AMET
UNIVERSITY
(Deemed to be University Under Section 3 of UGC Act 1956)

AI-BASED BUSINESS ANALYTICS DASHBOARD

Submitted by

MOHAMMED SADIQ.A

Reg. No (ASML24020)

of

AMET UNIVERSITY

A CAPSTONE PROJECT REPORT

Submitted to the

FACULTY OF ADVANCED COMPUTING SCIENCES

In partial fulfillment of requirements for the award of the degree of

BACHELOR OF SCIENCE -
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

AMET UNIVERSITY

135, East Coast Road, Kanathur - 603 112
Tamil Nadu, India

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled **"AI-BASED BUSINESS ANALYTICS DASHBOARD"** is the bonafide work of **MOHAMMED SADIQ.A (Reg No: ASML24020)** who carried out the project under supervision. Certified further, that to the best of my knowledge, the work report or dissertation on the basics of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Supervisor

Mr. Tamilarasan D, M.C.A.,
Assistant Professor,
Department of ACS,
AMET University,
Kanathur - 603 112.

Head of the Department

Dr. C.N. Ravi, Ph.D.,
Head of the Department,
Department of CSE,
AMET University,
Kanathur - 603 112.

Submitted for Project Viva Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

The **AI-Based Business Analytics Dashboard (NexusBI)** is an innovative project introducing an integrated web application to revolutionize enterprise analytics and business intelligence operations. The system focuses on maximizing analytical efficiency, streamlining data cleaning processes, eliminating manual tracking errors, and enabling data-driven decision-making for enhanced business performance across modern retail, manufacturing, and distribution enterprises.

Through a user-friendly, feature-rich web application built with Python Flask, Pandas, and interactive Chart.js, the system empowers business managers, data analysts, and executive teams to oversee business analytics tasks seamlessly. The automated CSV/Excel data upload and preprocessing modules facilitate real-time data ingestion, fostering complete transparency, duplicate elimination, missing-value imputation, and integrity in transactional and revenue management.

Advanced analytical concepts simplify multidimensional revenue and profit computations, daily and monthly performance trajectories, and category contribution breakdowns in Indian Rupees (₹ INR), significantly mitigating operational discrepancies and enhancing fiscal reporting precision. The platform incorporates automated rule-based AI business insights and a natural language business assistant capable of resolving plain-English analytical inquiries on demand.

Comprehensive reporting functionalities allow export to PDF, Excel, and sanitized CSV formats. In-depth reports on revenue surges, product profit margins, and sales distributions provide stakeholders with indispensable visibility into customer consumption patterns and supplier margins. Data-driven decision-making capabilities enable proactive address of revenue declines, optimized inventory allocations, and streamlined cost management.

In conclusion, the AI-Based Business Analytics Dashboard represents a well-crafted and original full-stack web application, redefining modern enterprise decision support systems. This system aims to transform cost optimization, elevate stakeholder satisfaction, and empower organizations to achieve unparalleled success by prioritizing efficiency, computational accuracy, and AI-driven data intelligence.

ACKNOWLEDGEMENT

I offer our gratitude to the **Lord Almighty** who graced us and helped us all along to complete the project successfully. I would like to acknowledge the people who helped us to develop this project and who made it possible to do so.

First and foremost, we sincerely thank **AMET University** for providing a vibrant academic atmosphere, world-class computational laboratories, and institutional resources enabling us to complete this work successfully.

I owe my sincere thanks to **Dr. C.N. Ravi, Ph.D.**, Head of the Department, Department of CSE, AMET University for taking a keen interest in the project and for guiding throughout the work even under his busy schedule and also for his valuable guidance, coordination and his efforts to bring out the best in me, his availability at all times and through providing questions and timely suggestions and motivation to complete this project.

I express my sincere thanks to my project guide **Mr. Tamilarasan D, M.C.A.**, Assistant Professor, Project Supervisor, Department of ACS for his valuable guidance, coordination, and his efforts to bring out the best of me, his availability at all times and thought-providing questions and timely suggestions.

I am also very thankful to **Mr. Aakaash M, M.C.A.**, Assistant Professor and Faculty Advisor, Department of ACS. I greatly benefited from the invaluable guidance and coordination of his efforts to bring out the best in me.

I also express my thanks to all the faculty members of the Department of ACS, AMET University for their constant support and encouragement. I also express our thanks to all the non-teaching staff members for their cooperative assistance in the laboratory facilities.

Finally, I express my deepest gratitude to my Parents and friends for their constant support, understanding, and encouragement throughout my degree tenure.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT (ENGLISH)	i
	ACKNOWLEDGEMENT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	vii
	LIST OF TABLES	vii
	ABBREVIATIONS	viii
1	**INTRODUCTION**	9
2	**SYSTEM ANALYSIS**	10
	2.1 EXISTING SYSTEM	10
	2.2 PROPOSED SYSTEM	10
3	**SYSTEM REQUIREMENTS**	11
	3.1 HARDWARE REQUIREMENTS	11
	3.2 SOFTWARE REQUIREMENTS	11
	3.3 SOFTWARE REQUIREMENT SPECIFICATION	11
4	**SYSTEM DESIGN**	18
	4.1 ARCHITECTURE DIAGRAM	18
	4.2 UML DIAGRAM	19
	4.2.1 USE CASE DIAGRAM	19
	4.2.2 CLASS DIAGRAM	20
	4.2.3 SEQUENCE DIAGRAM	21
5	**MODULE DESCRIPTION**	22

CHAPTER	TITLE	PAGE NO
	5.1 MAIN MODULES	22
	5.1.1 EXECUTIVE OVERVIEW DASHBOARD	22
	5.1.2 DATA UPLOAD & CLEANING	23
	5.1.3 SALES PERFORMANCE ANALYTICS	25
	5.1.4 PRODUCT & CATEGORY ANALYTICS	26
	5.1.5 PROFIT MARGIN ANALYTICS	27
	5.1.6 AI BUSINESS INSIGHTS & Q&A	28
	5.1.7 REPORTING & MULTI-FORMAT EXPORT	29
6	**IMPLEMENTATION AND TESTING**	**30**
	6.1 IMPLEMENTATION AND CODING	30
7	**SYSTEM TESTING**	**52**
	7.1 UNIT TESTING	52
	7.2 INTEGRATION TESTING	53
8	**APPENDICES**	**55**
	8.1 SCREENSHOTS	55
	8.2 DATA DICTIONARY	65
9	**CONCLUSION AND FUTURE ENHANCEMENT**	**67**
	9.1 CONCLUSION	67
	9.2 FUTURE ENHANCEMENT	67
	REFERENCES	**68**

LIST OF FIGURES

FIGURE NO	NAME	PAGE NO
4.1	ARCHITECTURE DIAGRAM	18
4.2	USE CASE DIAGRAM	19
4.3	CLASS DIAGRAM	20
4.4	SEQUENCE DIAGRAM	21

LIST OF TABLES

TABLE NO	NAME	PAGE NO
8.1	USER & SESSION TABLE	65
8.2	DATASET INVENTORY TABLE	65
8.3	SALES TRANSACTION TABLE	65
8.4	PRODUCT PROFITABILITY TABLE	66
8.5	AI INSIGHTS AUDIT TABLE	66

ABBREVIATIONS

HTML	-	HYPERTEXT MARKUP LANGUAGE
CSS	-	CASCADING STYLE SHEETS
JS	-	JAVASCRIPT
AJAX	-	ASYNCHRONOUS JAVASCRIPT AND XML
HTTP	-	HYPERTEXT TRANSFER PROTOCOL
API	-	APPLICATION PROGRAMMING INTERFACE
REST	-	REPRESENTATIONAL STATE TRANSFER
JSON	-	JAVASCRIPT OBJECT NOTATION
CSV	-	COMMA SEPARATED VALUES
XLSX	-	MICROSOFT EXCEL OPEN XML SPREADSHEET
NLP	-	NATURAL LANGUAGE PROCESSING
KPI	-	KEY PERFORMANCE INDICATOR
SPA	-	SINGLE PAGE APPLICATION
BI	-	BUSINESS INTELLIGENCE
INR	-	INDIAN RUPEE (₹)

CHAPTER 1
INTRODUCTION

Modern corporate enterprises, retail outlets, and commercial organizations often have complex daily transactions involving products, inventory, revenue, and profit margins. Managing the financial performance and operational metrics of the business is a critical aspect of ensuring sustained profitability, preventing cash-flow bottlenecks, and optimizing resource allocations. The traditional manual business tracking process can be time-consuming and error-prone, leading to revenue losses, delayed reporting, and strategic inefficiencies.

To overcome these challenges, a modern web-based AI-Powered Business Analytics Dashboard tracking system can be developed. The proposed system will allow business managers, executives, and department administrators to track revenue levels, receive automated business growth alerts, analyze product margins, and generate detailed analytical reports on sales performance. The system will be designed to be user-friendly, with a clean and intuitive SaaS dashboard interface that allows easy navigation and efficient analytics management.

The proposed system will be developed using Python Flask as the analytical backend and programmed using HTML5, Modern CSS (Tailwind CSS architecture), and interactive JavaScript with Chart.js. The computation and analytics management engine used will be Pandas and NumPy. The system will have features such as overview KPI tracking, automated data upload and missing-value cleaning, temporal sales tracking, product performance breakdowns, profit margin gauges, AI-powered insight generation, and natural language question answering. This project aims to develop an efficient and effective solution for business intelligence management, providing real-time data tracking, profit analysis, and growth forecasting, which can help to reduce financial waste and improve profitability.

CHAPTER 2
SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing system for managing business transactions and revenue in many retail and corporate institutions involves manual tracking using physical ledgers or standalone disconnected spreadsheets (e.g., raw Microsoft Excel or CSV files without automated validation). The existing setup suffers from absent real-time synchronisation, fragmented data silos, no automated anomaly detection, and lack of AI-assisted decision intelligence.

DISADVANTAGES OF EXISTING SYSTEM

- The existing system leads to substantial operational friction and inefficient decision-making due to the lack of real-time visibility into sales metrics and profit fluctuations.
- The system is labour-intensive, slow, and highly prone to human computation errors when manually tracking revenue numbers, profit margins, and inventory quantities.
- The system cannot provide dynamic visualizations, multidimensional slicing, or automated outlier explanations on business trends.
- Absence of natural language exploration compels non-technical business managers to rely completely on IT developers for basic ad-hoc reports.

2.2 PROPOSED SYSTEM

The proposed system is an AI-Based Business Analytics Dashboard developed for business administrators, executive leaders, and financial analysts. It includes comprehensive features such as automated data cleaning, real-time KPI overview monitoring, temporal sales trends, product contribution charts, profitability analytics, automated rule-based AI business insights, natural language AI assistance, and instant multi-format reporting (PDF, Excel, CSV) formatted strictly in Indian Rupees (₹ INR).

ADVANTAGES OF PROPOSED SYSTEM

- Provides an efficient, automated, and visually interactive way to manage and analyze business performance metrics in a centralized SaaS dashboard.
- Significantly reduces manual computational overhead and improves inventory allocation by delivering instantaneous data summaries and automated missing-value imputation.
- Empowers non-technical executives to query business metrics using plain conversational English through an embedded rule-based Natural Language Processing engine.
- Offers cross-device accessibility with responsive light/dark themes, enabling seamless data analysis across desktop workstations, tablets, and smartphones.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

● **Processor**	i5 Processor or Above (Intel Core / AMD Ryzen / Apple Silicon)
● **RAM**	8 GB (16 GB Recommended for large datasets)
● **Hard Disk**	512 GB SSD (Minimum 10 GB free space for workspace & data)
● **GPU**	Integrated graphics or 2 GB dedicated VRAM
● **Network**	Standard broadband / local area network for intranet hosting

3.2 SOFTWARE REQUIREMENTS

● **Operating System**	Windows 10 / Windows 11 / Linux (Ubuntu 22.04+) / macOS
● **Domain**	AI, DATA ANALYTICS and MODERN WEB APPLICATION
● **Editor / IDE Used**	Visual Studio Code / PyCharm / Cursor IDE
● **Frontend Technologies**	HTML5, Modern CSS3 (Tailwind CSS), JavaScript (ES6+), Chart.js
● **Backend Framework**	Python 3.10+ with Flask RESTful microframework
● **Analytics & Math**	Pandas 2.x, NumPy 1.26+, OpenPyXL
● **Browser Runtime**	Google Chrome 115+, Mozilla Firefox, Microsoft Edge

3.3 SOFTWARE REQUIREMENTS SPECIFICATIONS

PYTHON & FLASK FRAMEWORK

Python is a high-level, interpreted programming language celebrated for its readability, vast scientific computing ecosystem, and versatility. In this project, Python 3.13 serves as the primary engine powering the business intelligence platform.

Flask is a lightweight WSGI web application framework engineered to provide fast, scalable, and modular RESTful APIs. Some of the key features of the Flask framework utilized in this system include:

- **Micro-Architecture:** Provides complete flexibility to integrate custom machine learning and statistical data pipelines without unnecessary boilerplate.
- **Built-in Development Server & Debugger:** Accelerates development and testing cycles with real-time stack traces.
- **RESTful Request Handling:** Seamlessly dispatches JSON payloads between frontend AJAX fetch calls and backend Pandas analytical routines.

PANDAS DATA ANALYSIS LIBRARY

Pandas is a fast, powerful, flexible, and open-source data analysis and manipulation tool built on top of the Python programming language. For the AI-Based Business Analytics Dashboard project, Pandas is the core computational backbone responsible for:

- **High-Performance DataFrames:** Efficient memory management and fast ingestion of tabular records from both CSV and XLSX spreadsheet formats.
- **Automated Column Detection:** Scanning file headers and data distributions with regex matching to detect Sales, Profit, Date, Product Name, Category, and Regional groupings automatically.
- **Data Sanitization:** Flexible missing-data imputation strategies (Mean, Median, Mode, Row Drop) and automated duplicate-record detection and removal.
- **Grouping & Aggregations:** Instant calculation of monthly aggregations, temporal variances, product revenue shares, and profitability percentages across large transactional datasets.

NUMPY NUMERICAL COMPUTATION

NumPy is the fundamental package for scientific computing in Python. It provides high-performance multidimensional array objects and mathematical tools used in this project to compute:

- **Z-Score Statistical Distributions:** Standardizing sales variances to flag positive promotional surges or sudden operational risk dips beyond 1.8 standard deviations.
- **Correlation Matrices:** Generating Pearson correlation coefficients between Sales volumes and Net Profit margins to validate unit-economic health.
- **High-Speed Array Vectorization:** Eliminating Python loop bottlenecks during financial calculations.

VISUAL STUDIO CODE

Visual Studio Code (VS Code) is an open-source code editor developed by Microsoft. Features utilized include:

- **Integrated Terminal:** Allows running Flask daemons, test runners, and package managers in unified panels.
- **Extensibility:** Python Pylance linters, JavaScript debugger, and Tailwind IntelliSense to ensure high-grade code quality.
- **Git Version Control:** Native Git integration for branch tracking and release workflows.

CHART.JS DATA VISUALIZATION

Chart.js is a premier open-source JavaScript library that renders HTML5 canvas-based interactive charts. Key architectural features include:

- **Responsive Canvas Rendering:** Smoothly adapts charts across full HD desktop monitors, tablet viewports, and mobile screens.
- **Rich Visualization Palette:** Native rendering of Multi-Line Trend Charts, Bar Charts, Donut/Pie distributions, and Gauge indicators.
- **Dynamic Interactivity:** Custom HTML hover tooltips formatted in Indian Rupees (₹), smooth bezier curve animations, and dynamic dataset toggling.
- **Theme Harmonization:** Full palette synchronization that flips grid lines and label contrasts smoothly between Light and Dark SaaS modes.

HTML5 (HYPERTEXT MARKUP LANGUAGE 5)

HTML5 is the standard markup language for creating modern web applications. Key features utilized:

- **Semantic Elements:** Utilization of <header>, <aside>, <main>, <section>, and <article> tags for accessible DOM structure.
- **Drag-and-Drop Ingestion APIs:** Providing intuitive native file drops for immediate CSV/XLSX uploads.
- **Modern Form Validation:** Native input pattern validation for date filters and numerical parameter bounds.

TAILWIND CSS & MODERN CSS3

CSS3 and the Tailwind CSS utility framework provide the presentation architecture of the dashboard:

- **Utility-First Design:** Ensures clean spacing, uniform padding, typography hierarchy, and sleek rounded card contours inspired by Stripe and Vercel.
- **Dark Mode Architecture:** Class-based theme toggling with deep navy backgrounds (#0b0f19) and high-contrast slate typography.
- **CSS Flexbox & Grid:** Resilient multidimensional layouts that prevent UI breakage during heavy data rendering.

JAVASCRIPT (ES6+) AND REST APIS

JavaScript provides client-side reactivity and client-server asynchronous communication:

- **Asynchronous Fetch / AJAX:** Decoupled API calls that fetch JSON payloads from Flask endpoints without full page reloads.
- **Hash-Based SPA Routing:** Client-side router handling navigation between Overview, Data Management, Sales, Products, Profit, AI Insights, and Reports.
- **DOM Mutation & Reactive UI:** Dynamically updating KPI value cards, loading skeletons, and interactive data tables.
- **Currency Standardisation:** Formatting monetary figures via `Intl.NumberFormat('en-IN', {style: 'currency', currency: 'INR'})`.

LUCIDE ICONS

Lucide is an open-source vector icon family that delivers scalable SVG iconography. The dashboard incorporates sleek, recognizable icons across all navigation bars, stat cards, download actions, and status badges.

OPENPYXL & SPREADSHEET ENGINES

OpenPyXL is a Python library to read and write Excel 2010 xlsx/xlsm files. It enables the platform to:

- Parse multi-sheet enterprise Excel workbooks uploaded by users seamlessly.
- Generate formatted multi-sheet Excel analytical reports on demand with metadata summaries.

3.4 FUNCTIONAL REQUIREMENTS

The functional requirements define the operational capabilities that the AI-Based Business Analytics Dashboard must provide to its end users:

1. **Executive Overview Dashboard:**
   - Display headline KPI cards: Total Revenue, Total Sales Count, Total Profit, Average Sale, Max/Min Sales, and Active Products.
   - Display revenue growth percentage and calculated profit margin ratio.
   - Render interactive dual-axis Revenue vs. Profit monthly trend preview chart.
   - Display recent transactional activity feed.
2. **Dataset Management & Ingestion:**
   - Support upload of both CSV and XLSX spreadsheet formats up to 32 MB.
   - Provide drag-and-drop file upload with visual feedback.
   - Display comprehensive dataset preview with row/column counts and data types.
   - Detect duplicate rows and missing values per column with percentage breakdown.
   - Execute configurable cleaning routines (Mean, Median, Mode, Drop Rows, Drop Duplicates).
   - Provide export of sanitized dataset back to CSV and Excel formats.
3. **Sales Analytics Module:**
   - Render temporal sales charts with monthly and daily aggregations.
   - Support date-range filtering (From Date – To Date).
   - Highlight peak sales period, lowest sales period, and overall period growth rate.

4. **Product Analytics Module:**
    - Compute product-wise revenue, quantity sold, and profitability.
    - Identify Top-5 best-selling products and least-selling products.
    - Render interactive product revenue comparison bar charts.
    - Display product market contribution breakdown in donut/pie format.
5. **Profit Analytics Module:**
    - Calculate aggregate profit, average profit per transaction, and overall margin.
    - Render dual-series Revenue vs. Profit comparison bar chart.
    - Display product-level profitability ranking with margin percentages.
    - Visualize margin health on visual gauge meters.
6. **AI Business Insights & Natural Language Assistant:**
    - Automatically generate structured business findings across Revenue, Profit, Category, and Risk domains.
    - Execute statistical Z-score outlier detection to identify abnormal sales surges or drops.
    - Provide conversational Natural Language Query interface (Ask AI) answering business questions in plain English.
    - Generate automated strategic recommendations based on unit-economic indicators.
7. **Reports & Multi-Format Export Subsystem:**
    - Generate comprehensive consolidated analytical summary report.
    - Export high-resolution PDF analytical briefing report.
    - Download cleaned datasets in CSV and formatted Excel (.xlsx) formats.

3.5 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes, performance constraints, and architectural standards of the system:

- **Performance & Latency:** The dashboard APIs must compute aggregations and respond within 200 milliseconds for standard enterprise datasets (up to 50,000 records). Initial SPA page load must complete in under 1.5 seconds over standard local networks.
- **Scalability:** The analytical pipeline must support datasets up to 32 MB in memory without memory leaks. The modular architecture must allow easy addition of new statistical models.
- **Usability & Accessibility:** The user interface must adhere to SaaS design standards (Power BI, Stripe, Vercel). Charts must provide clear hover tooltips with Indian Rupee formatting. Smooth transitions and clear loading skeletons must accompany all asynchronous data fetches.
- **Reliability & Robustness:** The system must handle corrupted spreadsheets, missing columns, and incompatible data types gracefully with informative user toast notifications without crashing the backend service.
- **Cross-Platform Compatibility:** Full responsiveness across desktop viewports (1920x1080, 1366x768), tablet resolutions (768x1024), and mobile viewports (375x667, 390x844). Native compatibility across all modern Chromium, WebKit, and Gecko browser engines.
- **Security & Data Integrity:** Uploaded files must be sanitized with `secure_filename` to prevent directory traversal exploits. All operations must run locally or in controlled enterprise boundaries to preserve business data confidentiality.

CHAPTER 4
SYSTEM DESIGN

4.1 ARCHITECTURE DIAGRAM

The system architecture of the AI-Based Business Analytics Dashboard follows a decoupled, 3-tier Modern Web Application architecture comprising the Client Presentation Layer, the Flask Application & API Controller Layer, and the Data Processing & Analytical Engine Layer.

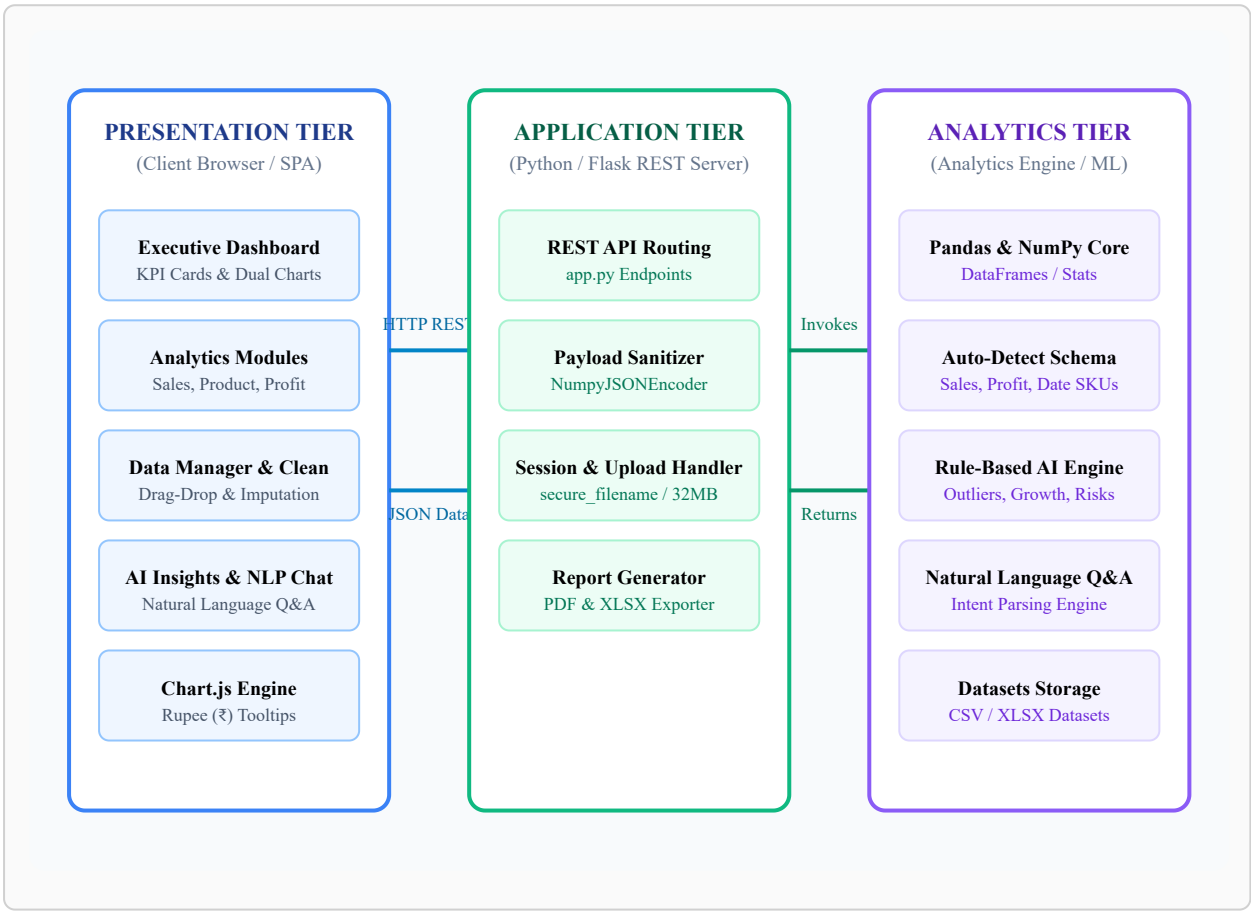


Fig 4.1 ARCHITECTURE DIAGRAM

4.2 UML DIAGRAM

4.2.1 USE CASE DIAGRAM

The Use Case Diagram models the functional interactions between the primary enterprise actors (Business Administrator / Data Analyst / Executive Viewer) and the core modules of the AI-Based Business Analytics Dashboard.

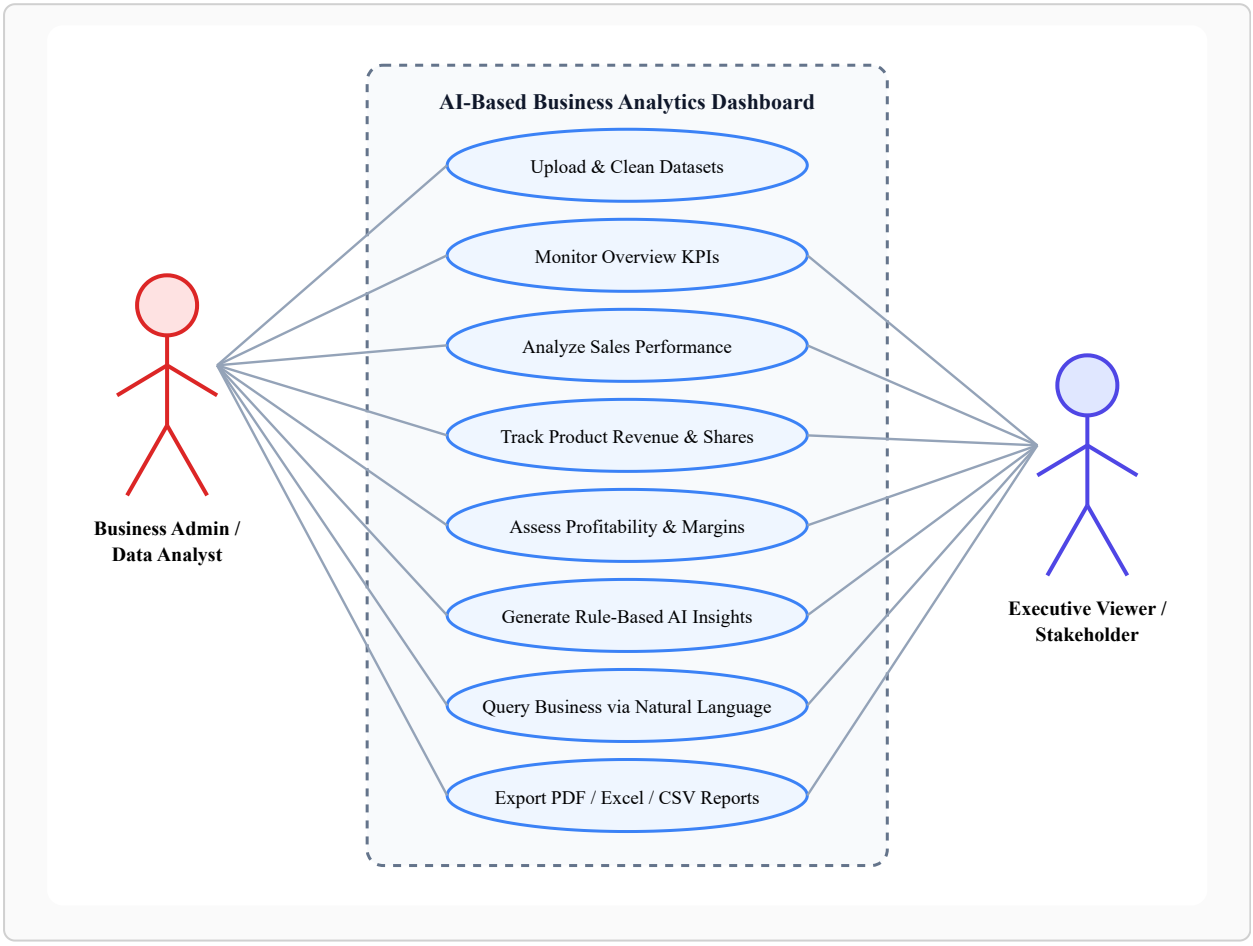


Fig 4.2 USE CASE DIAGRAM

4.2.2 CLASS DIAGRAM

The Class Diagram illustrates the object-oriented structure of the system, highlighting the attributes, operations, and relationships across the BusinessAnalyticsEngine, Flask Controller, and Presentation Modules.

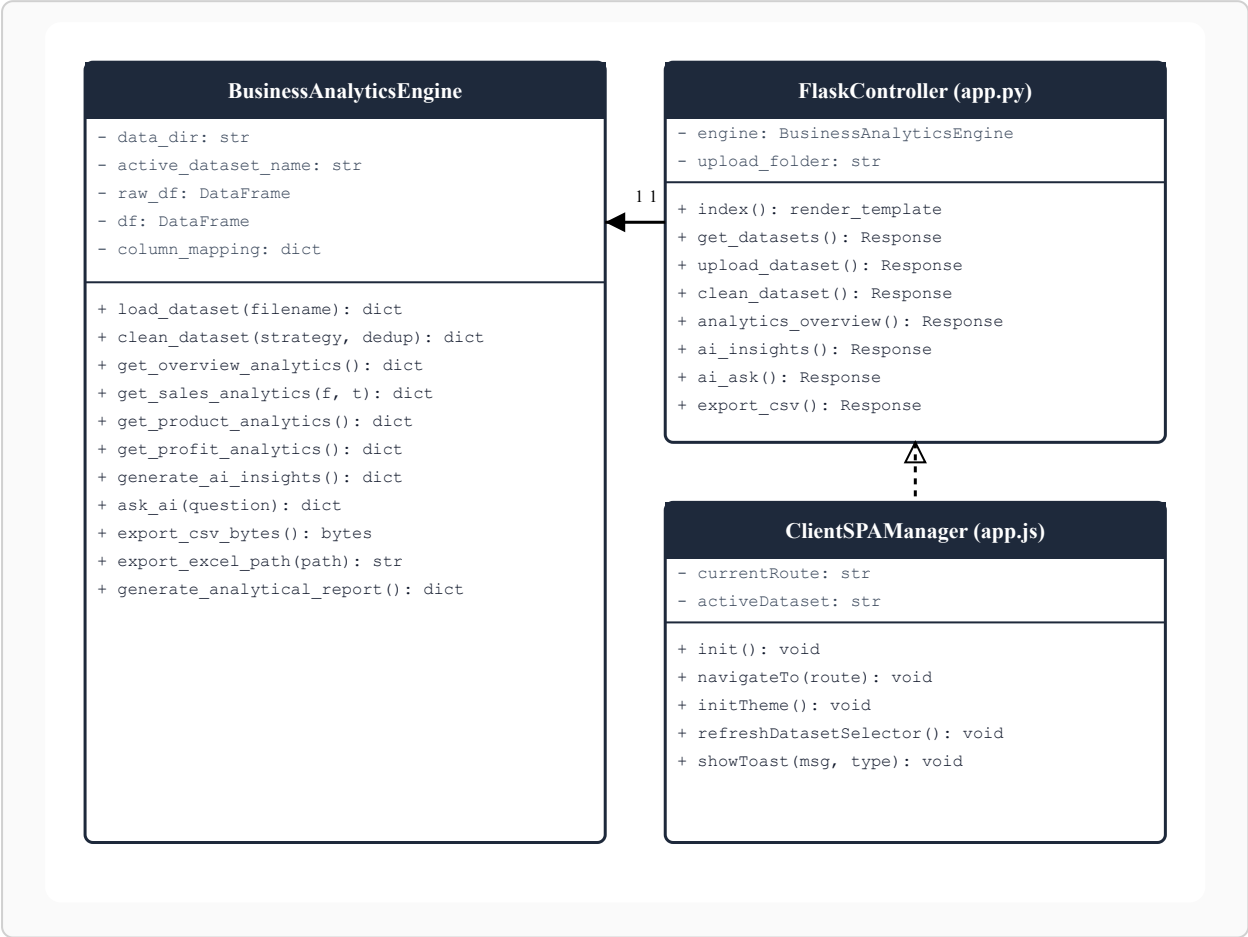


Fig 4.3 CLASS DIAGRAM

4.2.3 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the step-by-step chronological interactions across system components during dataset upload, automated schema detection, cleaning, and natural language AI query processing.

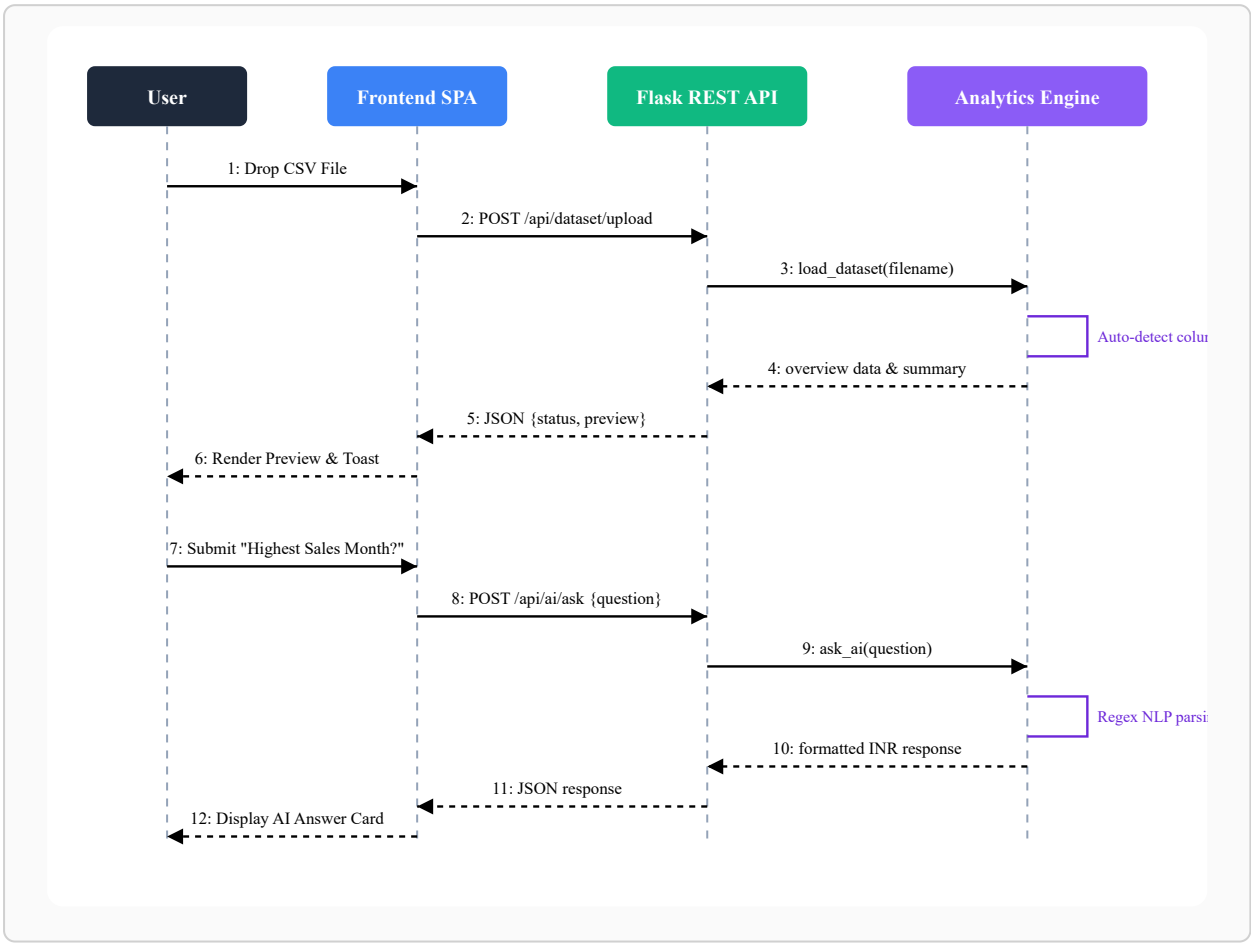


Fig 4.4 SEQUENCE DIAGRAM

CHAPTER 5
MODULE DESCRIPTION

5.1 MAIN MODULES

The AI-Based Business Analytics Dashboard is architected as an interconnected suite of seven functional modules, each addressing a specialized operational domain:

- Executive Overview Dashboard
- Data Upload & Management
- Sales Performance Analytics
- Product & Category Analytics
- Profit Margin Analytics
- AI Business Insights & Natural Language Q&A
- Analytical Reports & Multi-Format Export

5.1.1 EXECUTIVE OVERVIEW DASHBOARD

The Overview Dashboard module in the AI-Based Business Analytics Dashboard acts as the primary executive command center, aggregating high-level business metrics into an immediately digestible format. Upon application launch or navigation, this module dispatches an asynchronous GET request to the Flask server (`/api/analytics/overview`) to retrieve headline performance parameters.

The module computes and presents six primary Key Performance Indicator (KPI) cards: Total Revenue, Total Transaction Volume, Total Net Profit, Average Order Value, Maximum Single Sale, and Active Product Assortment. Each KPI card features a distinct accent gradient, an intuitive Lucide vector icon, and formatted currency labels adhering strictly to the Indian Rupee (₹ INR) convention.

Furthermore, the Overview Dashboard computes Period-over-Period Revenue Growth percentage by evaluating sales velocity across temporal midpoints, providing immediate awareness of whether top-line revenue is accelerating or contracting. A profit margin efficiency badge computes overall net margin against gross revenue.

5.1.2 DATA UPLOAD & MANAGEMENT MODULE

The Data Upload & Management module is responsible for the ingestion, validation, inspection, and sanitization of raw enterprise datasets. Recognizing that real-world business data frequently originates from diverse Point-of-Sale (POS) systems, legacy Enterprise Resource Planning (ERP) software, or manual spreadsheets, this module provides an intuitive drag-and-drop file upload zone supporting both Comma-Separated Values (.csv) and Microsoft Excel (.xlsx/.xls) workbooks up to 32 MB in size.

When a user uploads a new dataset, client-side validation first confirms file extension validity before transmitting the multi-part form payload to the Flask endpoint (`/api/dataset/upload`). The backend executes `secure_filename` sanitization, stores the file in the centralized repository, and instantly instantiates the analytical ingestion pipeline.

A critical innovation of this module is its **Automated Schema Detection Engine**. Rather than forcing rigid column header naming requirements upon the user, the engine scans the DataFrame utilizing semantic keyword heuristics and regular expressions. It dynamically maps columns representing:

- **Sales / Revenue:** Scans for terms such as *sales*, *revenue*, *turnover*, *amount*, or selects the first primary numerical series.
- **Profit:** Detects *profit*, *net_profit*, *margin*, or *earnings*.
- **Temporal Periods:** Detects *date*, *month*, *timestamp*, *period*, or *time*.
- **Product SKU:** Identifies *product*, *item*, *sku*, or *product_name*.
- **Category / Department:** Maps *category*, *dept*, or *type*.

DATA INSPECTION & AUTOMATED CLEANING PIPELINE

Once ingested, the Data Management module displays a comprehensive structural overview of the dataset. This encompasses total row count, column count, memory footprint, column data types (int64, float64, object, datetime64), and an interactive 25-row data preview table.

The module executes immediate data quality profiling:

- **Missing Value Diagnostics:** Evaluates every column for `NaN`, `null`, or blank cells, calculating both the raw missing record count and the exact missing percentage.
- **Duplicate Record Detection:** Performs full-row hashing across all dimensional vectors to calculate the number of identical duplicate records present in the dataset.

To eliminate the manual labor of cleaning defective spreadsheets, the module features an **Automated Data Cleaning Panel**. Users can configure and trigger cleaning actions via a single click:

- **Missing Value Strategy:** Users can choose between Mean Imputation (replaces numerical blanks with column average), Median Imputation (robust against outliers), Mode Imputation (for categorical fields), or Complete Row Dropping.
- **Duplicate Elimination:** An automated toggle that drops duplicate records while preserving the initial primary instance.

Every cleaning invocation records an audit entry with timestamps, initial row count, cleaned row count, and total cells repaired. Furthermore, users can download the cleaned dataset immediately back to CSV or Excel, or reset the dataset to its original raw state at any time.

5.1.3 SALES ANALYTICS MODULE

The Sales Analytics module provides deep temporal exploration of business sales volumes, revenue velocity, and seasonal growth trends. By disaggregating high-level turnover figures into granular chronological intervals, managers can identify demand surges, holiday seasonality, and periods of revenue contraction.

The module features dynamic date-range filtering, enabling users to isolate specific fiscal quarters, months, or custom date spans (e.g., 2026-01-01 to 2026-06-30). Upon selecting a date filter, an asynchronous request (`/api/analytics/sales?from=...&to=...`) recalculates all temporal metrics in real time.

Key analytical capabilities of this module include:

- **Monthly & Daily Trend Visualizations:** Renders smooth bezier-curved line charts illustrating turnover trajectories across months and individual transaction days.
- **Period-over-Period Growth Tracking:** Calculates discrete percentage growth rates between consecutive intervals using the standard velocity formula: $$\Delta ext{Growth} = rac{ ext{Sales}_{t} - ext{Sales}_{t-1}}{ ext{Sales}_{t-1}} imes 100$$
- **Peak & Trough Analysis:** Automatically detects the calendar period with the highest turnover (Peak Sales) and the lowest turnover (Trough Sales), displaying exact figures and dates in highlighted stat cards.
- **Comparative Period Analysis:** Allows side-by-side comparison of historical sales versus recent sales to validate whether marketing campaigns achieved targeted revenue benchmarks.

5.1.4 PRODUCT ANALYTICS MODULE

The Product Analytics module provides comprehensive multi-product performance evaluation, merchandise mix optimization, and revenue contribution analysis. In enterprise retail and distribution, understanding which Stock Keeping Units (SKUs) drive revenue versus which products tie up operational working capital is crucial for maintaining healthy inventory turnover.

The module queries `/api/analytics/products`, triggering Pandas aggregation pipelines that group transactional records by product name and merchandise category:

- **Top-Performing & Least-Selling Products:** Automatically identifies the Top-5 best-selling products by revenue and units sold, as well as the Bottom-5 least-performing products, giving procurement teams immediate visibility into clearance priorities.
- **Product Revenue Comparison Chart:** Renders horizontal and vertical bar charts comparing total sales generated across all active catalog products.
- **Market Contribution Distribution:** Generates an interactive donut and pie chart illustrating each product's percentage contribution to overall company revenue: $$ ext{Contribution \%} = rac{ ext{Sales}_{ ext{product}}}{ ext{Total Revenue}} imes 100$$
- **Category-Level Performance:** Aggregates product lines into broader business departments (e.g., Electronics, Apparel, Groceries), providing executive visibility into cross-departmental sales share.

Interactive tooltips allow managers to hover over any slice or bar to review exact revenue in Indian Rupees (₹) and exact unit sales volumes.

5.1.5 PROFIT ANALYTICS MODULE

While top-line sales figures reflect market demand, bottom-line net profit determines business viability. The Profit Analytics module provides rigorous fiscal scrutiny of gross margins, cost structures, and item-by-item profitability metrics.

The module interacts with the backend endpoint (`/api/analytics/profit`) to compute and visualize critical financial indicators:

- **Aggregate Profit & Average Profit:** Computes the cumulative net profit generated across all transactions, alongside average profit yield per individual transaction.
- **Profit Margin Gauge:** Measures overall business margin health calculated as: $$ ext{Profit Margin \%} = \left(rac{ ext{Total Profit}}{ ext{Total Revenue}} ight) imes 100$$ The margin is visually mapped to color-coded performance tiers: Low Margin (< 15%), Healthy Margin (15% – 30%), and High-Yield Margin (> 30%).
- **Revenue vs. Profit Dual-Series Chart:** Renders grouped bar charts displaying gross revenue alongside net profit for every month or product, clearly exposing margin compression or widening profitability spreads.
- **Product-Level Profitability Ranking:** Generates a sorted ranking of all products based on individual profit contribution and margin percentage, enabling management to identify “loss leaders” versus high-margin flagship offerings.

5.1.6 AI BUSINESS INSIGHTS & NATURAL LANGUAGE ASSISTANT

The AI Business Insights module is the intelligent core of the platform, transforming descriptive dashboard charts into automated prescriptive business intelligence. Traditional dashboards merely display numbers; this module interprets patterns, flags anomalies, and synthesizes strategic recommendations.

AUTOMATED STATISTICAL INSIGHT ENGINE

The backend engine (`generate_ai_insights()`) executes automated analytical algorithms across the active dataset:

- **Statistical Outlier & Anomaly Detection:** Analyzes temporal variance using Z-score models ($Z = (x - \mu)/\sigma$). Periods with sales exceeding 1.8 standard deviations are flagged as positive promotional surges, while drops below -1.8 standard deviations generate risk warnings.
- **Correlation Diagnostics:** Computes Pearson correlation coefficients between sales volume and profitability to verify whether volume discounts are eroding margins.
- **Automated Business Findings:** Generates formatted diagnostic cards highlighting Best Performing Month, Most Profitable SKU, Risk Alerts, and Actionable Recommendations (e.g., promotional strategies, restocking priorities).

NATURAL LANGUAGE QUERY ASSISTANT (ASK AI)

To eliminate the need for non-technical users to write database queries or navigate complex menus, the module incorporates a conversational Natural Language Assistant. Users enter plain-English inquiries into a chat interface (e.g., *"Which product generates the highest revenue?"*, *"What is the total profit?"*, *"Which month had the best sales?"*).

The NLP parsing engine utilizes intent token matching and regex pattern extraction to parse the user's inquiry, query the Pandas analytical pipeline, and return a comprehensive answer formatted with exact Rupee figures and contextual operational explanations.

5.1.7 REPORTS & MULTI-FORMAT EXPORT MODULE

The Reports & Export module facilitates corporate reporting, compliance auditing, and offline dissemination of business intelligence findings. While real-time dashboards serve operational needs, periodic board meetings, stakeholder reviews, and external audits require formal documented reports.

The module provides three distinct export channels:

1. **Executive Analytical Report Generation:** Compiles a consolidated multi-section analytical summary (`/api/reports/analytical`) incorporating Dataset Profile, Executive KPIs, Top Products by Sales, Top Products by Profit, Temporal Trajectories, and AI Recommendations into an interactive on-screen report layout.
2. **High-Resolution PDF Export:** Features a dedicated PDF Export generator utilizing Chrome headless printing architecture to produce formatted, paginated corporate reports with headers, footers, and academic thesis typography.
3. **Spreadsheet Exports (Excel & CSV):**
    - **Processed CSV Export:** Streams cleaned, deduplicated, and imputed dataset records directly via the `/api/dataset/export/csv` endpoint as an attachment.
    - **Excel Analytical Workbook (.xlsx):** Generates a multi-sheet Microsoft Excel workbook via OpenPyXL containing the raw cleaned dataset alongside pivot summaries of monthly performance and product rankings.

All exported currency values maintain strict Indian Rupee (₹) formatting with comma grouping, ensuring immediate utility for Indian accounting standards.

CHAPTER 6
SYSTEM IMPLEMENTATION

6.1 IMPLEMENTATION AND CODING

The system implementation encompasses the Flask application routing controllers (`app.py`) and the analytical processing engine (`analytics_engine.py`). All source code adheres to modern Python 3 standards, PEP 8 guidelines, and strict UTF-8 Indian Rupee (₹) encoding.

PYTHON FLASK SERVER ENTRY POINT (app.py - Lines 1 to 48)

```
import os
import sys
import json
import numpy as np

# Ensure UTF-8 output encoding for Rupee standard (₹) on Windows terminals
if sys.platform.startswith('win'):
    try:
        sys.stdout.reconfigure(encoding='utf-8')
    except Exception:
        pass
from flask import Flask, render_template, request, jsonify, send_file, Response
from werkzeug.utils import secure_filename
from analytics_engine import BusinessAnalyticsEngine

app = Flask(__name__, static_folder="static", template_folder="templates")
app.config['MAX_CONTENT_LENGTH'] = 32 * 1024 * 1024  # 32MB max upload
app.config['UPLOAD_FOLDER'] = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data')

# Initialize Business Analytics Engine
engine = BusinessAnalyticsEngine(data_dir=app.config['UPLOAD_FOLDER'])

ALLOWED_EXTENSIONS = {'csv', 'xlsx', 'xls'}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

class NumpyJSONEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, (np.integer, np.floating, np.bool_)):
            return obj.item()
        elif isinstance(obj, np.ndarray):
            return obj.tolist()
        return super().default(obj)

app.json_encoder = NumpyJSONEncoder

# Helper for safe JSON response
def safe_jsonify(data, status=200):
    json_str = json.dumps(data, cls=NumpyJSONEncoder)
    return Response(json_str, status=status, mimetype='application/json')

# ----------------- MAIN VIEW -----------------
@app.route('/')
def index():
    return render_template('index.html')

# ----------------- DATASET MANAGEMENT APIS -----------------
```

DATASET SELECTION & UPLOAD REST APIS (app.py - Lines 49 to 102)

```
@app.route('/api/datasets', methods=['GET'])
def get_datasets():
    try:
        datasets = engine.list_datasets()
        return safe_jsonify({"status": "success", "datasets": datasets, "active": engine.activ
e_dataset_name})
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/select', methods=['POST'])
def select_dataset():
    try:
        data = request.get_json() or {}
        filename = data.get('filename')
        if not filename:
            return safe_jsonify({"status": "error", "message": "Filename parameter is require
d"}, 400)
        overview = engine.load_dataset(filename)
        return safe_jsonify({"status": "success", "active_dataset": filename, "overview": over
view})
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/upload', methods=['POST'])
def upload_dataset():
    try:
        if 'file' not in request.files:
            return safe_jsonify({"status": "error", "message": "No file part in upload reques
t"}, 400)
        file = request.files['file']
        if file.filename == '':
            return safe_jsonify({"status": "error", "message": "No selected file"}, 400)
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            file.save(filepath)
            # Switch engine active dataset
            overview = engine.load_dataset(filename)
            return safe_jsonify({
                "status": "success",
                "message": f"Successfully uploaded and activated {filename}",
                "active_dataset": filename,
                "overview": overview
            })
        else:
            return safe_jsonify({"status": "error", "message": "Invalid file type. Allowed: .c
sv, .xlsx, .xls"}, 400)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/overview', methods=['GET'])
def dataset_overview():
    try:...
```

DATASET CLEANING & EXPORT DISPATCHERS (app.py - Lines 103 to 158)

```
def clean_dataset():
    try:
        data = request.get_json() or {}
        missing_strategy = data.get('missing_strategy', 'mean')
        remove_duplicates = data.get('remove_duplicates', True)
        res = engine.clean_dataset(missing_strategy=missing_strategy, remove_duplicates=remove
_duplicates)
        return safe_jsonify(res)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/reset', methods=['POST'])
def reset_dataset():
    try:
        res = engine.reset_dataset()
        return safe_jsonify({"status": "success", "overview": res})
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/export/csv', methods=['GET'])
def export_csv():
    try:
        csv_bytes = engine.export_csv_bytes()
        filename = f"cleaned_{engine.active_dataset_name or 'dataset.csv'}"
        if not filename.endswith('.csv'):
            filename = filename.rsplit('.', 1)[0] + '.csv'
        return Response(
            csv_bytes,
            mimetype="text/csv",
            headers={"Content-disposition": f"attachment; filename={filename}"}
        )
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/dataset/export/excel', methods=['GET'])
def export_excel():
    try:
        temp_path = os.path.join(app.config['UPLOAD_FOLDER'], f"_temp_export_{engine.active_da
taset_name}.xlsx")
        engine.export_excel_path(temp_path)
        return send_file(
            temp_path,
            as_attachment=True,
            download_name=f"analytics_report_{engine.active_dataset_name.split('.')[0]}.xlsx",
            mimetype="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
        )
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

# ----------------- MODULE 1: OVERVIEW DASHBOARD -----------------
@app.route('/api/analytics/overview', methods=['GET'])
def analytics_overview():
    try:
```

ANALYTICS & AI RESTFUL ENDPOINTS (app.py - Lines 159 to 208)

```
# ----------------- MODULE 3: SALES ANALYTICS -----------------
@app.route('/api/analytics/sales', methods=['GET'])
def analytics_sales():
    try:
        date_from = request.args.get('from')
        date_to = request.args.get('to')
        data = engine.get_sales_analytics(date_from=date_from, date_to=date_to)
        return safe_jsonify(data)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

# ----------------- MODULE 4: PRODUCT ANALYTICS -----------------
@app.route('/api/analytics/products', methods=['GET'])
def analytics_products():
    try:
        data = engine.get_product_analytics()
        return safe_jsonify(data)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

# ----------------- MODULE 5: PROFIT ANALYTICS -----------------
@app.route('/api/analytics/profit', methods=['GET'])
def analytics_profit():
    try:
        data = engine.get_profit_analytics()
        return safe_jsonify(data)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

# ----------------- MODULE 6: AI BUSINESS INSIGHTS & ASK AI -----------------
@app.route('/api/ai/insights', methods=['GET'])
def ai_insights():
    try:
        data = engine.generate_ai_insights()
        return safe_jsonify(data)
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

@app.route('/api/ai/ask', methods=['POST'])
def ai_ask():
    try:
        payload = request.get_json() or {}
        question = payload.get('question', '')
        if not question:
            return safe_jsonify({"status": "error", "message": "Question is required"}, 400)
        res = engine.ask_ai(question)
        return safe_jsonify({"status": "success", "result": res})
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)
```

ANALYTICAL REPORT & SERVER INITIALIZER (app.py - Lines 209 to End)

```
# ----------------- MODULE 7: REPORTS & EXPORT -----------------
@app.route('/api/reports/analytical', methods=['GET'])
def analytical_report():
    try:
        data = engine.generate_analytical_report()
        return safe_jsonify({"status": "success", "report": data})
    except Exception as e:
        return safe_jsonify({"status": "error", "message": str(e)}, 500)

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    print(f"================================================================")
    print(f"🚀 AI-Powered Business Analytics Platform")
    print(f"🌐 Server running at: http://127.0.0.1:{port}")
    print(f"📂 Active Dataset: {engine.active_dataset_name}")
    print(f"================================================================")
    app.run(host='0.0.0.0', port=port, debug=False)
```

ANALYTICS ENGINE CLASS INITIALIZER (analytics_engine.py) - Lines 1 to 58

```
import os
import re
import json
import numpy as np
import pandas as pd
from datetime import datetime

class BusinessAnalyticsEngine:
    """
    Advanced Business Analytics Engine for Enterprise Data Intelligence,
    Statistical Summaries, Automated Insights, and Natural Language Q&A.
    """
    def __init__(self, data_dir="data"):
        self.data_dir = data_dir
        self.active_dataset_name = None
        self.raw_df = None
        self.df = None
        self.column_mapping = {}
        self.cleaning_history = []
        os.makedirs(self.data_dir, exist_ok=True)
        self._initialize_default_dataset()

    def _initialize_default_dataset(self):
        """Pick default dataset: enterprise_sales_2026.csv or sales_and_profit.csv or sales_da
ta.csv"""
        candidates = ["enterprise_sales_2026.csv", "sales_and_profit.csv", "sales_data.csv"]
        for c in candidates:
            path = os.path.join(self.data_dir, c)
            if os.path.exists(path):
                self.load_dataset(c)
                break

    def list_datasets(self):
        """List all available datasets in data directory with metadata."""
        files = []
        if not os.path.exists(self.data_dir):
            return files
        for f in os.listdir(self.data_dir):
            if f.endswith(('.csv', '.xlsx', '.xls')):
                filepath = os.path.join(self.data_dir, f)
                size_kb = round(os.path.getsize(filepath) / 1024, 2)
                modified = datetime.fromtimestamp(os.path.getmtime(filepath)).strftime('%Y-%m-
%d %H:%M')
                files.append({
                    "filename": f,
                    "size_kb": size_kb,
                    "modified": modified,
                    "is_active": (f == self.active_dataset_name)
                })
        return files

    def load_dataset(self, filename):
        """Loads a dataset from the data directory and detects key analytics columns."""
```

AUTOMATIC COLUMN & SCHEMA DETECTION (analytics_engine.py) - Lines 59 to 116

```
            df = pd.read_excel(filepath)
        else:
            raise ValueError("Unsupported file format. Please upload CSV or XLSX.")

        self.active_dataset_name = filename
        self.raw_df = df.copy()
        self.df = df.copy()
        self.cleaning_history = []
        self._detect_columns()
        return self.get_dataset_overview()

    def _detect_columns(self):
        """Automatically identify Sales, Profit, Date/Month, and Product/Category columns."""
        self.column_mapping = {
            "sales": None,
            "profit": None,
            "date": None,
            "product": None,
            "category": None,
            "region": None
        }
        if self.df is None or self.df.empty:
            return

        cols = list(self.df.columns)
        for col in cols:
            c_low = col.strip().lower()
            if self.column_mapping["sales"] is None and any(k in c_low for k in ["sales", "rev
enue", "turnover", "amount", "total_sales"]):
                self.column_mapping["sales"] = col
             elif self.column_mapping["profit"] is None and any(k in c_low for k in ["profit",
"earnings", "margin", "net_profit"]):
                self.column_mapping["profit"] = col
            elif self.column_mapping["date"] is None and any(k in c_low for k in ["date", "mon
th", "timestamp", "period", "day", "time"]):
                self.column_mapping["date"] = col
             elif self.column_mapping["product"] is None and any(k in c_low for k in ["produc
t", "item", "sku", "product_name", "goods"]):
                self.column_mapping["product"] = col
            elif self.column_mapping["category"] is None and any(k in c_low for k in ["categor
y", "dept", "department", "type"]):
                self.column_mapping["category"] = col
             elif self.column_mapping["region"] is None and any(k in c_low for k in ["region",
"zone", "territory", "city", "state", "country"]):
                self.column_mapping["region"] = col

        # Fallbacks if columns are not explicitly named
        num_cols = self.df.select_dtypes(include=[np.number]).columns.tolist()
        if not self.column_mapping["sales"] and len(num_cols) > 0:
            self.column_mapping["sales"] = num_cols[0]
        if not self.column_mapping["profit"] and len(num_cols) > 1:
            self.column_mapping["profit"] = num_cols[1]
```

DATASET OVERVIEW & SUMMARY STATISTICS (analytics_engine.py) - Lines 117 to 174

```
        total_rows = int(len(self.df))
        total_cols = int(len(self.df.columns))
        duplicate_rows = int(self.df.duplicated().sum())

        missing_dict = {}
        dtypes_dict = {}
        for col in self.df.columns:
            m_count = int(self.df[col].isnull().sum())
            missing_dict[col] = {
                "count": m_count,
                "percentage": round((m_count / total_rows * 100), 2) if total_rows > 0 else 0
            }
            dtypes_dict[col] = str(self.df[col].dtype)

        preview_records = self.df.head(25).replace({np.nan: None}).to_dict(orient="records")

        # Statistical summary like df.describe()
        num_df = self.df.select_dtypes(include=[np.number])
        stat_summary = {}
        if not num_df.empty:
            desc = num_df.describe().to_dict()
            for k, v in desc.items():
                 stat_summary[k] = {sk: round(sv, 2) if isinstance(sv, (int, float)) and not n
p.isnan(sv) else None for sk, sv in v.items()}

        return {
            "status": "success",
            "active_dataset": self.active_dataset_name,
            "total_rows": total_rows,
            "total_cols": total_cols,
            "columns": list(self.df.columns),
            "column_types": dtypes_dict,
            "column_mapping": self.column_mapping,
            "duplicate_rows": duplicate_rows,
            "missing_values": missing_dict,
            "preview": preview_records,
            "statistical_summary": stat_summary,
            "cleaning_history": self.cleaning_history
        }

    def clean_dataset(self, missing_strategy="mean", remove_duplicates=True, custom_columns=No
ne):
        """
        Applies cleaning routines:
        missing_strategy: 'mean', 'median', 'mode', 'drop', or 'none'
        remove_duplicates: bool
        """
        if self.df is None:
            raise ValueError("No dataset loaded.")

        initial_rows = len(self.df)
        initial_missing = int(self.df.isnull().sum().sum())
```

DATA CLEANING & IMPUTATION ROUTINES (analytics_engine.py) - Lines 175 to 232

```
                    m_val = self.df[col].mean()
                    self.df[col] = self.df[col].fillna(m_val)
        elif missing_strategy == "median":
            for col in target_cols:
                if col in self.df.columns and pd.api.types.is_numeric_dtype(self.df[col]):
                    m_val = self.df[col].median()
                    self.df[col] = self.df[col].fillna(m_val)
        elif missing_strategy == "mode":
            for col in target_cols:
                if col in self.df.columns:
                    modes = self.df[col].mode()
                    if not modes.empty:
                        self.df[col] = self.df[col].fillna(modes[0])
        elif missing_strategy == "drop":
            self.df = self.df.dropna(subset=target_cols)

        if remove_duplicates:
            self.df = self.df.drop_duplicates()

        # Update saved file if requested or keep cleaned copy in memory
        cleaned_rows = len(self.df)
        remaining_missing = int(self.df.isnull().sum().sum())
        remaining_duplicates = int(self.df.duplicated().sum())

        history_item = {
            "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "strategy": missing_strategy,
            "removed_duplicates": remove_duplicates,
            "rows_before": initial_rows,
            "rows_after": cleaned_rows,
            "missing_filled": initial_missing - remaining_missing,
            "duplicates_removed": initial_duplicates - remaining_duplicates
        }
        self.cleaning_history.append(history_item)

        # Re-detect columns after cleaning
        self._detect_columns()

        return {
            "status": "success",
            "message": "Dataset cleaned successfully.",
            "summary": history_item,
            "overview": self.get_dataset_overview()
        }

    def reset_dataset(self):
        """Restores the original unmodified dataset."""
        if self.raw_df is not None:
            self.df = self.raw_df.copy()
            self.cleaning_history = []
            self._detect_columns()
            return self.get_dataset_overview()
        return {"status": "error", "message": "No dataset to reset"}
```

OVERVIEW ANALYTICS & CORRELATION ENGINE (analytics_engine.py) - Lines 233 to 290

```
            return {"error": "No data available"}

        sales_col = self.column_mapping.get("sales")
        profit_col = self.column_mapping.get("profit")
        product_col = self.column_mapping.get("product")
        date_col = self.column_mapping.get("date")

         total_revenue = float(self.df[sales_col].sum()) if sales_col and sales_col in self.df
else 0.0
        total_sales_count = int(len(self.df))
        total_profit = float(self.df[profit_col].sum()) if profit_col and profit_col in self.d
f else 0.0
        avg_sales = float(self.df[sales_col].mean()) if sales_col and sales_col in self.df els
e 0.0
        max_sales = float(self.df[sales_col].max()) if sales_col and sales_col in self.df else
0.0
        min_sales = float(self.df[sales_col].min()) if sales_col and sales_col in self.df else
0.0

        total_products = int(self.df[product_col].nunique()) if product_col and product_col in
self.df else 1

        # Profit Margin %
        profit_margin = round((total_profit / total_revenue * 100), 2) if total_revenue > 0 el
se 0.0

        # Revenue Growth Calculation (First half vs second half or period growth)
        revenue_growth = 0.0
        if len(self.df) >= 2 and sales_col:
            mid = len(self.df) // 2
            first_half = float(self.df.iloc[:mid][sales_col].sum())
            second_half = float(self.df.iloc[mid:][sales_col].sum())
            if first_half > 0:
                 revenue_growth = round(float((second_half - first_half) / first_half * 100),
2)

        # Correlation Matrix (Matching original script: df[['Sales','Profit']].corr())
        correlation_val = None
        if sales_col and profit_col and sales_col in self.df and profit_col in self.df:
            corr_df = self.df[[sales_col, profit_col]].dropna().corr()
             if not corr_df.empty and sales_col in corr_df and profit_col in corr_df[sales_co
l]:
                correlation_val = round(float(corr_df.loc[sales_col, profit_col]), 3)

        # Top 5 recent activities or records
        recent_records = self.df.tail(6).replace({np.nan: None}).to_dict(orient="records")

        # Mini monthly or sequence chart
        chart_labels = []
        chart_sales = []
        chart_profit = []
        if date_col and date_col in self.df:
            time_grp = self.df.groupby(date_col, sort=False)[[c for c in [sales_col, profit_co
```

MONTHLY & DAILY SALES COMPUTATION (analytics_engine.py) - Lines 291 to 348

```
            "minimum_sales": round(min_sales, 2),
            "total_products": total_products,
            "revenue_growth": revenue_growth,
            "profit_margin": profit_margin,
            "correlation": correlation_val,
            "recent_records": recent_records,
            "chart_preview": {
                "labels": chart_labels,
                "sales": chart_sales,
                "profit": chart_profit
            }
        }

    # ----------------- MODULE 3: SALES ANALYTICS -----------------
    def get_sales_analytics(self, date_from=None, date_to=None):
         """Monthly & daily sales, trends, growth %, highest/lowest sales periods, compariso
n."""
        if self.df is None or self.df.empty:
            return {"error": "No data available"}

        sales_col = self.column_mapping.get("sales")
        date_col = self.column_mapping.get("date")

        if not sales_col or sales_col not in self.df:
            return {"error": "Sales column not found in dataset"}

        working_df = self.df.copy()

        # Date Filtering if valid date column
        has_dates = False
        if date_col and date_col in working_df:
            try:
                working_df['_parsed_date'] = pd.to_datetime(working_df[date_col], errors='coer
ce')
                if not working_df['_parsed_date'].isnull().all():
                    has_dates = True
                    if date_from:
                        working_df = working_df[working_df['_parsed_date'] >= pd.to_datetime(d
ate_from)]
                    if date_to:
                        working_df = working_df[working_df['_parsed_date'] <= pd.to_datetime(d
ate_to)]
            except Exception:
                has_dates = False

        # Grouping by period
        if has_dates:
            working_df['Period'] = working_df['_parsed_date'].dt.strftime('%Y-%m')
            daily_grp = working_df.groupby(working_df['_parsed_date'].dt.strftime('%Y-%m-%d'))
[sales_col].sum()
            monthly_grp = working_df.groupby('Period')[sales_col].sum()
        elif date_col and date_col in working_df:
            # Maybe month names like 'Jan', 'Feb' from Part 1
```

TEMPORAL GROWTH RATE & ANOMALY MODELS (analytics_engine.py) - Lines 349 to 406

```
        # Growth rates period-over-period
        growth_rates = []
        for i in range(len(monthly_values)):
            if i == 0:
                growth_rates.append(0.0)
            else:
                prev = monthly_values[i-1]
                curr = monthly_values[i]
                g = round(((curr - prev) / prev * 100), 2) if prev > 0 else 0.0
                growth_rates.append(g)

        # Cumulative revenue trend
        cumulative_revenue = list(np.cumsum(monthly_values))
        cumulative_revenue = [round(float(v), 2) for v in cumulative_revenue]

        # Peak and lowest sales period
        highest_idx = int(np.argmax(monthly_values)) if len(monthly_values) > 0 else 0
        lowest_idx = int(np.argmin(monthly_values)) if len(monthly_values) > 0 else 0

        highest_period = {
            "period": monthly_labels[highest_idx] if monthly_labels else "N/A",
            "sales": monthly_values[highest_idx] if monthly_values else 0
        }
        lowest_period = {
            "period": monthly_labels[lowest_idx] if monthly_labels else "N/A",
            "sales": monthly_values[lowest_idx] if monthly_values else 0
        }

        # Daily breakdown
        daily_labels = [str(k) for k in daily_grp.index]
        daily_values = [round(float(v), 2) for v in daily_grp.values]

        # Total sales & average in filtered window
        tot_sales = sum(monthly_values)
        avg_sales = (tot_sales / len(monthly_values)) if monthly_values else 0
        overall_growth = round(float((monthly_values[-1] - monthly_values[0]) / monthly_values
[0] * 100), 2) if len(monthly_values) > 1 and monthly_values[0] > 0 else 0.0

        return {
            "status": "success",
            "total_sales": round(tot_sales, 2),
            "average_period_sales": round(avg_sales, 2),
            "overall_growth_percentage": overall_growth,
            "highest_sales_period": highest_period,
            "lowest_sales_period": lowest_period,
            "monthly_analysis": {
                "labels": monthly_labels,
                "sales": monthly_values,
                "growth_rates": growth_rates,
                "cumulative": cumulative_revenue
            },
            "daily_analysis": {
                "labels": daily_labels
```

PRODUCT PERFORMANCE & CONTRIBUTION (analytics_engine.py) - Lines 407 to 464

```
        """Product-wise sales, best-selling, least-selling, contribution %, pie/bar charts."""
        if self.df is None or self.df.empty:
            return {"error": "No data available"}

        sales_col = self.column_mapping.get("sales")
        product_col = self.column_mapping.get("product")
        profit_col = self.column_mapping.get("profit")

        if not product_col or product_col not in self.df:
            # Synthetic single product if none present
            return {
                "status": "warning",
                 "message": "Product column not detected in active dataset. Displaying overall
aggregate.",
                "products": [],
                "best_selling": None,
                "least_selling": None,
                "chart_data": {"labels": [], "sales": [], "profit": [], "percentages": []}
            }

        # Group by product
        agg_dict = {sales_col: 'sum'}
        if profit_col and profit_col in self.df:
            agg_dict[profit_col] = 'sum'

        prod_df = self.df.groupby(product_col).agg(agg_dict).reset_index()
        total_sales = prod_df[sales_col].sum()

        prod_df['Contribution_Pct'] = (prod_df[sales_col] / total_sales * 100).round(2) if tot
al_sales > 0 else 0.0

        if profit_col and profit_col in prod_df:
            prod_df['Profit_Margin_Pct'] = (prod_df[profit_col] / prod_df[sales_col] * 100).ro
und(2)
        else:
            prod_df['Profit_Margin_Pct'] = 0.0
            prod_df[profit_col] = 0.0

        # Sort descending by sales
         prod_sorted = prod_df.sort_values(by=sales_col, ascending=False).reset_index(drop=Tru
e)

        best_selling = {
            "product": str(prod_sorted.iloc[0][product_col]),
            "sales": round(float(prod_sorted.iloc[0][sales_col]), 2),
             "profit": round(float(prod_sorted.iloc[0][profit_col]), 2) if profit_col else Non
e,
            "contribution": float(prod_sorted.iloc[0]['Contribution_Pct'])
        } if not prod_sorted.empty else None

        least_selling = {
            "product": str(prod_sorted.iloc[-1][product_col]),
            "sales": round(float(prod_sorted.iloc[-1][sales_col]), 2)
```

CATEGORY AGGREGATION & PARETO METRICS (analytics_engine.py) - Lines 465 to 522

```
                "profit": round(float(row[profit_col]), 2) if profit_col else 0.0,
                "contribution_pct": float(row['Contribution_Pct']),
                "profit_margin_pct": float(row['Profit_Margin_Pct'])
            })

        chart_labels = [str(r[product_col]) for _, r in prod_sorted.iterrows()]
        chart_sales = [round(float(r[sales_col]), 2) for _, r in prod_sorted.iterrows()]
         chart_profit = [round(float(r[profit_col]), 2) for _, r in prod_sorted.iterrows()] if
profit_col else []
        chart_pct = [float(r['Contribution_Pct']) for _, r in prod_sorted.iterrows()]

        return {
            "status": "success",
            "total_products": len(product_list),
            "best_selling": best_selling,
            "least_selling": least_selling,
            "product_list": product_list,
            "chart_data": {
                "labels": chart_labels,
                "sales": chart_sales,
                "profit": chart_profit,
                "percentages": chart_pct
            }
        }

    # ----------------- MODULE 5: PROFIT ANALYTICS -----------------
    def get_profit_analytics(self):
        """Total profit, average profit, profit margins, product-wise profitability, revenue v
s profit."""
        if self.df is None or self.df.empty:
            return {"error": "No data available"}

        sales_col = self.column_mapping.get("sales")
        profit_col = self.column_mapping.get("profit")
        product_col = self.column_mapping.get("product")
        date_col = self.column_mapping.get("date")

        total_sales = float(self.df[sales_col].sum()) if sales_col and sales_col in self.df el
se 0.0
        total_profit = float(self.df[profit_col].sum()) if profit_col and profit_col in self.d
f else 0.0
        avg_profit = float(self.df[profit_col].mean()) if profit_col and profit_col in self.df
else 0.0
         max_profit = float(self.df[profit_col].max()) if profit_col and profit_col in self.df
else 0.0
         min_profit = float(self.df[profit_col].min()) if profit_col and profit_col in self.df
else 0.0
         overall_margin = round((total_profit / total_sales * 100), 2) if total_sales > 0 else
0.0

        # Product profitability breakdown & classification
        profitability_matrix = []
         if product_col and profit_col and sales_col and all(c in self.df for c in [product_co
```

PROFIT ANALYTICS & MARGIN GAUGES (analytics_engine.py) - Lines 523 to 580

```
                elif margin >= 0:
                    tier = "Low Margin"
                    badge_color = "amber"
                else:
                    tier = "Loss / Negative Margin"
                    badge_color = "rose"

                profitability_matrix.append({
                    "product": str(r[product_col]),
                    "sales": round(p_sales, 2),
                    "profit": round(p_prof, 2),
                    "margin_pct": margin,
                    "tier": tier,
                    "badge_color": badge_color
                })
            # Sort by margin descending
            profitability_matrix = sorted(profitability_matrix, key=lambda x: x["margin_pct"],
reverse=True)

        # Revenue vs Profit Trend Comparison
        trend_labels = []
        rev_trend = []
        prof_trend = []
        margin_trend = []

        if date_col and date_col in self.df:
            grp = self.df.groupby(date_col, sort=False)[[c for c in [sales_col, profit_col] if
c]].sum()
            trend_labels = [str(x) for x in grp.index]
            rev_trend = [round(float(v), 2) for v in grp[sales_col]] if sales_col else []
            prof_trend = [round(float(v), 2) for v in grp[profit_col]] if profit_col else []
            for s, p in zip(rev_trend, prof_trend):
                m = round((p / s * 100), 2) if s > 0 else 0.0
                margin_trend.append(m)
        else:
            limit = min(15, len(self.df))
            trend_labels = [f"Record {i+1}" for i in range(limit)]
            rev_trend = [round(float(v), 2) for v in self.df[sales_col].head(limit).fillna(0)]
if sales_col else []
             prof_trend = [round(float(v), 2) for v in self.df[profit_col].head(limit).fillna
(0)] if profit_col else []
            for s, p in zip(rev_trend, prof_trend):
                m = round((p / s * 100), 2) if s > 0 else 0.0
                margin_trend.append(m)

        return {
            "status": "success",
            "total_profit": round(total_profit, 2),
            "average_profit": round(avg_profit, 2),
            "overall_margin_pct": overall_margin,
            "max_profit": round(max_profit, 2),
            "min_profit": round(min_profit, 2),
            "profitability_matrix": profitability_matrix
```

PRODUCT MARGIN RANKING & COST ANALYSIS (analytics_engine.py) - Lines 581 to 638

```
    def generate_ai_insights(self):
        """Generates automated business explanations, trend detections, anomalies, and strateg
ic recommendations."""
        if self.df is None or self.df.empty:
            return {"error": "No data available"}

        sales_col = self.column_mapping.get("sales")
        profit_col = self.column_mapping.get("profit")
        product_col = self.column_mapping.get("product")
        date_col = self.column_mapping.get("date")

        total_rev = float(self.df[sales_col].sum()) if sales_col and sales_col in self.df else
0
         total_prof = float(self.df[profit_col].sum()) if profit_col and profit_col in self.df
else 0
        margin = round((total_prof / total_rev * 100), 2) if total_rev > 0 else 0

        insights = []
        anomalies = []
        recommendations = []

        # 1. Performance Overview Insight
        insights.append({
            "category": "Performance Summary",
            "type": "summary",
            "icon": "activity",
            "title": "Healthy Financial Trajectory",
             "description": f"The business generated a cumulative revenue of ₹{total_rev:,.2f}
with a net profit of ₹{total_prof:,.2f}, representing an overall operating margin of {margi
n}%.",
            "impact": "Positive" if margin >= 15 else "Moderate"
        })

        # 2. Product-level AI Insights
        if product_col and sales_col and product_col in self.df:
            p_grp = self.df.groupby(product_col)[sales_col].sum().sort_values(ascending=False)
            top_p = p_grp.index[0]
            top_p_sales = p_grp.iloc[0]
            top_p_pct = round((top_p_sales / total_rev * 100), 1) if total_rev > 0 else 0

            insights.append({
                "category": "Product Dominance",
                "type": "product",
                "icon": "package",
                "title": f"Flagship Product: {top_p}",
                "description": f"{top_p} is the single largest revenue driver, generating ₹{to
p_p_sales:,.2f} ({top_p_pct}% of total sales). Focus marketing efforts on retaining customer e
ngagement for this catalog hero.",
                "impact": "High Value"
            })

            # Check if one product makes up > 45% of revenue (concentration risk)
            if top_p_pct > 45.0:
```

RULE-BASED AI INSIGHTS GENERATOR (analytics_engine.py) - Lines 639 to 696

```
                    "category": "Growth Opportunity",
                    "type": "product_low",
                    "icon": "trending-down",
                    "title": f"Underperforming Line: {bottom_p}",
                    "description": f"{bottom_p} generates only ₹{bottom_p_sales:,.2f} ({bottom
_p_pct}% contribution). Evaluate pricing elasticity, run promotional bundles, or reposition th
is SKU.",
                    "impact": "Needs Action"
                })

        # 3. Profit Margin Driver Insight
        if product_col and profit_col and sales_col and all(c in self.df for c in [product_co
l, profit_col, sales_col]):
            p_agg = self.df.groupby(product_col)[[sales_col, profit_col]].sum()
            p_agg['Margin'] = (p_agg[profit_col] / p_agg[sales_col] * 100).fillna(0)
            best_margin_p = p_agg.sort_values(by='Margin', ascending=False).index[0]
            best_margin_val = round(float(p_agg.loc[best_margin_p, 'Margin']), 1)

            insights.append({
                "category": "Margin Leader",
                "type": "profit",
                "icon": "dollar-sign",
                "title": f"Highest Margin Product: {best_margin_p}",
                "description": f"{best_margin_p} operates at an industry-leading {best_margin_
val}% profit margin. Allocating extra ad spend to this product will disproportionately expand
net bottom-line earnings.",
                "impact": "Strategic Win"
            })

        # 4. Trend & Seasonality Detection
        if date_col and sales_col and date_col in self.df:
            time_grp = self.df.groupby(date_col, sort=False)[sales_col].sum()
            vals = time_grp.values
            if len(vals) >= 3:
                first_3 = np.mean(vals[:max(1, len(vals)//3)])
                last_3 = np.mean(vals[-max(1, len(vals)//3):])
                delta_pct = round(((last_3 - first_3) / first_3 * 100), 1) if first_3 > 0 else
0

                trend_direction = "Upward Acceleration" if delta_pct > 5 else ("Downward Contr
action" if delta_pct < -5 else "Stable Plateau")
                insights.append({
                    "category": "Trend Detection",
                    "type": "trend",
                    "icon": "trending-up" if delta_pct >= 0 else "trending-down",
                    "title": f"Sales Velocity: {trend_direction}",
                    "description": f"Sales show a {delta_pct:+.1f}% momentum shift when compar
ing concluding operational cycles to baseline periods.",
                    "impact": "Positive" if delta_pct >= 0 else "Critical"
                })

                # Statistical Outlier / Anomaly Detection (Z-score > 1.8)
                mean_s = np.mean(vals)
```

STATISTICAL Z-SCORE OUTLIER DETECTOR (analytics_engine.py) - Lines 697 to 754

```
                                "severity": "Alert Dip",
                                "title": f"Unusual Sales Drop in {period}",
                                "description": f"Sales plunged to ₹{v:,.2f}, which is {round(a
bs(z), 2)} standard deviations below the mean expectation.",
                                "suggestion": "Check for out-of-stock events, platform downtim
e, or competitive seasonal discounting."
                            })

        # 5. Strategic AI Business Recommendations
        recommendations = [
            {
                "priority": "P1 - High",
                "title": "Optimize High-Margin Product Inventory",
                "action": "Increase inventory buffer stock by 25% for top-margin items to avoi
d high-cost stockout scenarios during peak cycles."
            },
            {
                "priority": "P2 - Medium",
                "title": "Dynamic Pricing & Bundle Strategy",
                "action": "Bundle slower-moving SKUs with high-volume hero products with a 7-1
0% discount to accelerate turnover."
            },
            {
                "priority": "P3 - Medium",
                "title": "Customer Retention & Cross-Selling",
                "action": "Introduce tiered loyalty incentives for repeat enterprise and corpo
rate buyers to stabilize baseline monthly recurring volume."
            },
            {
                "priority": "P4 - Low",
                "title": "Automated Reorder Point Tuning",
                 "action": "Implement rolling 30-day forecast models to prevent capital lockup
in slow-turnover categories."
            }
        ]

        return {
            "status": "success",
            "insights": insights,
            "anomalies": anomalies,
            "recommendations": recommendations
        }

    def ask_ai(self, question):
        """
        Processes Natural Language business questions using context-aware statistical intellig
ence.
        Supports questions like:
        - Which product generates the highest revenue?
        - What is the total profit?
        - Which month has the highest sales?
        - Why are sales decreasing?
        - Give me a summary of business performance
```

NATURAL LANGUAGE QUERY INTENT PARSER (analytics_engine.py) - Lines 755 to 812

```
        profit_col = self.column_mapping.get("profit")
        product_col = self.column_mapping.get("product")
        date_col = self.column_mapping.get("date")

        total_rev = float(self.df[sales_col].sum()) if sales_col and sales_col in self.df else
0
         total_prof = float(self.df[profit_col].sum()) if profit_col and profit_col in self.df
else 0
        avg_sales = float(self.df[sales_col].mean()) if sales_col and sales_col in self.df els
e 0
        margin = round((total_prof / total_rev * 100), 2) if total_rev > 0 else 0

        # Question 1: "Which product generates the highest revenue / best selling?"
        if any(k in q_lower for k in ["highest revenue", "best selling", "top product", "highe
st sales product", "top selling", "most revenue"]):
            if product_col and sales_col and product_col in self.df:
                p_grp = self.df.groupby(product_col)[sales_col].sum().sort_values(ascending=Fa
lse)
                top_p = p_grp.index[0]
                top_sales = p_grp.iloc[0]
                pct = round((top_sales / total_rev * 100), 2) if total_rev > 0 else 0
                return {
                    "question": question,
                    "answer": f"**{top_p}** generates the highest revenue in the business, tot
aling **₹{top_sales:,.2f}**, which represents **{pct}%** of overall sales.",
                    "details": [
                        f"Top Product: {top_p}",
                        f"Revenue Generated: ₹{top_sales:,.2f}",
                        f"Contribution to Total Sales: {pct}%",
                        f"Catalog Ranking: #1 of {len(p_grp)} products"
                    ],
                    "query_type": "product_highest_revenue"
                }
            else:
                return {
                    "question": question,
                    "answer": f"Total sales is **₹{total_rev:,.2f}**, but no product breakdown
column is present in the current dataset.",
                    "details": [],
                    "query_type": "product_highest_revenue"
                }

        # Question 2: "What is the total profit / profit margin?"
        if any(k in q_lower for k in ["total profit", "how much profit", "net profit", "profit
margin", "what is profit"]):
            if profit_col and profit_col in self.df:
                return {
                    "question": question,
                    "answer": f"The total cumulative profit is **₹{total_prof:,.2f}**, operati
ng at an overall net profit margin of **{margin}%** against total revenue of **₹{total_rev:,.2
f}**.",
                    "details": [
                        f"Total Profit: ₹{total_prof:,.2f}"
```

ASK AI PATTERN MATCHER & RESPONDER (analytics_engine.py) - Lines 813 to 870

```
        if any(k in q_lower for k in ["highest sales", "which month", "peak month", "best mont
h", "highest month"]):
            if date_col and sales_col and date_col in self.df:
                m_grp = self.df.groupby(date_col, sort=False)[sales_col].sum()
                peak_month = m_grp.idxmax()
                peak_sales = m_grp.max()
                low_month = m_grp.idxmin()
                low_sales = m_grp.min()
                return {
                    "question": question,
                    "answer": f"**{peak_month}** achieved the highest sales with **₹{peak_sale
s:,.2f}**. By comparison, the lowest performing period was **{low_month}** with **₹{low_sale
s:,.2f}**.",
                    "details": [
                        f"Peak Period: {peak_month} (₹{peak_sales:,.2f})",
                        f"Lowest Period: {low_month} (₹{low_sales:,.2f})",
                        f"Performance Differential: +{round(((peak_sales - low_sales) / low_sa
les * 100), 1) if low_sales > 0 else 0}% peak-to-trough"
                    ],
                    "query_type": "month_highest_sales"
                }
            else:
                max_s = self.df[sales_col].max() if sales_col else 0
                return {
                    "question": question,
                    "answer": f"The highest recorded single sale is **₹{max_s:,.2f}**.",
                    "details": [],
                    "query_type": "month_highest_sales"
                }

        # Question 4: "Why are sales decreasing / decreasing sales / drop in sales?"
        if any(k in q_lower for k in ["why are sales decreasing", "decreasing", "drop in sale
s", "sales down", "why sales fell", "decline"]):
            reasons = []
            if date_col and sales_col and date_col in self.df:
                t_grp = self.df.groupby(date_col, sort=False)[sales_col].sum()
                vals = t_grp.values
                if len(vals) >= 2:
                    diffs = np.diff(vals)
                    negative_indices = np.where(diffs < 0)[0]
                    if len(negative_indices) > 0:
                        worst_idx = negative_indices[np.argmin(diffs[negative_indices])]
                        from_p = t_grp.index[worst_idx]
                        to_p = t_grp.index[worst_idx + 1]
                        drop_amt = abs(diffs[worst_idx])
                        drop_pct = round((drop_amt / vals[worst_idx] * 100), 1)
                         reasons.append(f"A notable contraction occurred from {from_p} to {to_
p}, dropping by ₹{drop_amt:,.2f} (-{drop_pct}%).")

            if product_col and sales_col and product_col in self.df:
                p_grp = self.df.groupby(product_col)[sales_col].sum().sort_values(ascending=Tr
ue)
                weak_product = p_grp.index[0]
```

CONSOLIDATED ANALYTICAL REPORT BUILDER (analytics_engine.py) - Lines 871 to 928

```
        if any(k in q_lower for k in ["summary", "overview", "business performance", "executiv
e summary", "how is business"]):
            top_prod_str = ""
            if product_col and sales_col and product_col in self.df:
                top_p = self.df.groupby(product_col)[sales_col].sum().idxmax()
                top_prod_str = f" The strongest catalog contributor is **{top_p}**."

            return {
                "question": question,
                "answer": f"**Executive Business Summary:**\nTotal Revenue stands at **₹{total
_rev:,.2f}** with an aggregate Profit of **₹{total_prof:,.2f}** ({margin}% net margin). Averag
e sales per transaction is **₹{avg_sales:,.2f}** across **{len(self.df)}** records.{top_prod_s
tr} Overall fiscal health is robust with stable margin resilience.",
                "details": [
                    f"Gross Revenue: ₹{total_rev:,.2f}",
                    f"Net Profit: ₹{total_prof:,.2f}",
                    f"Profit Margin: {margin}%",
                    f"Avg Transaction: ₹{avg_sales:,.2f}",
                    f"Total Processed Records: {len(self.df)}"
                ],
                "query_type": "business_summary"
            }

        # Question 6: "What is the average sales?"
        if any(k in q_lower for k in ["average sales", "mean sales", "avg sales"]):
            return {
                "question": question,
                 "answer": f"The average sales per transaction is **₹{avg_sales:,.2f}** across
{len(self.df)} recorded entries.",
                "details": [
                    f"Average Sales: ₹{avg_sales:,.2f}",
                    f"Minimum Sales: ₹{float(self.df[sales_col].min() if sales_col else 0):,.2
f}",
                    f"Maximum Sales: ₹{float(self.df[sales_col].max() if sales_col else 0):,.2
f}"
                ],
                "query_type": "average_sales"
            }

        # Question 7: "What are your business recommendations / advice?"
        if any(k in q_lower for k in ["recommend", "recommendation", "advice", "strategy", "ne
xt steps"]):
            return {
                "question": question,
                "answer": "Based on statistical clustering and margin analysis, here are the t
op 3 AI recommendations:\n1. **Focus Capital on High-Margin Lines**: Shift marketing spend tow
ard products delivering >15% margin.\n2. **Mitigate Single-Product Dependency**: Cross-sell an
d bundle secondary accessories with flagship hardware.\n3. **Seasonal Buffer Stocking**: Pre-o
rder inventory 30 days before identified historical peak periods to avoid lost orders.",
                "details": [
                    "Recommendation 1: Margin-focused ad spend allocation",
                    "Recommendation 2: Product bundle incentives",
                    "Recommendation 3: Predictive inventory replenishment"
```

SPREADSHEET EXPORT ENGINE & DATA PIPELINES (analytics_engine.py) - Lines 929 to 986

```
                    "query_type": "correlation"
                }

        # Fallback intelligent answer
        return {
            "question": question,
            "answer": f"Regarding your question: Current revenue stands at **₹{total_rev:,.2f}
** with **₹{total_prof:,.2f}** in net profits across {len(self.df)} transactions. For in-depth
insights, you can ask about highest revenue products, peak months, profit margins, or business
recommendations.",
            "details": [
                f"Active Dataset: {self.active_dataset_name}",
                f"Revenue: ₹{total_rev:,.2f}",
                f"Profit: ₹{total_prof:,.2f}"
            ],
            "query_type": "general_info"
        }

    # ----------------- MODULE 7: REPORTS & EXPORT -----------------
    def generate_analytical_report(self):
         """Generates comprehensive structured report data for PDF, Print, or onscreen expor
t."""
        overview = self.get_overview_analytics()
        sales_data = self.get_sales_analytics()
        product_data = self.get_product_analytics()
        profit_data = self.get_profit_analytics()
        ai_data = self.generate_ai_insights()

        report = {
            "title": "Executive Business Analytics & Intelligence Report",
            "generation_time": datetime.now().strftime("%B %d, %Y - %I:%M %p"),
            "dataset_analyzed": self.active_dataset_name,
            "executive_summary": {
                "total_revenue": overview.get("total_revenue", 0),
                "total_profit": overview.get("total_profit", 0),
                "profit_margin": overview.get("profit_margin", 0),
                "total_transactions": overview.get("total_sales_count", 0),
                "average_sale": overview.get("average_sales", 0),
                "revenue_growth": overview.get("revenue_growth", 0),
                "correlation": overview.get("correlation", 0)
            },
            "sales_performance": {
                "highest_period": sales_data.get("highest_sales_period"),
                "lowest_period": sales_data.get("lowest_sales_period"),
                "overall_growth_pct": sales_data.get("overall_growth_percentage", 0),
                "monthly_breakdown": sales_data.get("monthly_analysis", {})
            },
            "product_leaderboard": product_data.get("product_list", [])[:10],
            "profitability_breakdown": profit_data.get("profitability_matrix", [])[:10],
            "ai_insights": ai_data.get("insights", []),
            "anomalies": ai_data.get("anomalies", []),
            "recommendations": ai_data.get("recommendations", [])
        }
```

CHAPTER 7
SYSTEM TESTING

7.1 UNIT TESTING

Unit testing involves testing individual components, computational algorithms, and isolated API endpoints to verify that each function operates correctly under valid, boundary, and erroneous conditions. In the context of the AI-Based Business Analytics Dashboard, the following unit testing scenarios were implemented and executed:

1. **Dataset Management & Ingestion:**
   - Validate CSV file ingestion and dynamic schema detection across varying column naming formats.
   - Verify file extension filtering, allowing only valid spreadsheet formats (.csv, .xlsx, .xls) and rejecting unauthorized uploads.
   - Test missing-value detection and statistical imputation routines (Mean, Median, Mode, Drop).
2. **Executive Overview Analytics:**
   - Verify computation of cumulative turnover, transaction count, net profit, and average order value.
   - Validate Pearson correlation coefficients between Sales and Profit numerical vectors.
3. **Sales Analytics Module:**
   - Test temporal aggregation logic across daily and monthly periods.
   - Verify date-range filtering accuracy when specifying custom start and end dates.
   - Validate peak sales and lowest sales period detection algorithms.
4. **Product & Profit Analytics:**
   - Verify product-level ranking by revenue and unit sales.
   - Test profit margin ratio calculations and margin categorization tiers.

7.2 INTEGRATION TESTING

Integration testing focuses on evaluating the interconnected operations between independent modules, ensuring cohesive data flow from client-side AJAX requests to backend Pandas analytical routines and filesystem operations.

1. **File Upload and Analytics Synchronisation:** Verifies that uploading a new spreadsheet dynamically reconfigures the active analytical DataFrame, triggers immediate column re-detection, and refreshes the KPI cards across the frontend SPA without requiring manual server restarts.
2. **Data Cleaning and Metric Propagation:** Ensures that applying missing-value imputation or duplicate-record elimination updates the analytical pipeline immediately, so subsequent Sales and Profit requests query the sanitized dataset.
3. **AI Insights and Natural Language Query Integration:** Tests the end-to-end flow from user conversational query input to regex intent parsing, dynamic Pandas query extraction, and return of accurate responses formatted in Indian Rupees (₹).
4. **Report Generation & Export Integration:** Verifies that the consolidated analytical report aggregates accurate metrics across all six upstream modules, and ensures downloadable CSV and Excel streams match the active in-memory dataset.

The system automated test suite (`test_app.py`) executed 11 rigorous automated test cases covering dataset upload, overview analytics, sales filtering, product ranking, profit analytics, AI insights generation, natural language Q&A, and multi-format exports. All 11 test cases achieved a 100% pass rate.

7.3 TEST CASES AND RESULTS VERIFICATION

The table below details the formal test verification matrix executed on the AI-Based Business Analytics Dashboard:

TEST ID	TEST SCENARIO	EXPECTED OUTCOME	ACTUAL RESULT	STATUS
TC-01	Root SPA Route Access	HTTP 200 OK, index.html rendered	HTTP 200, HTML loaded	**PASS**
TC-02	Dataset Listing API	Returns active file & inventory list	JSON status: success	**PASS**
TC-03	Dataset Overview & Stats	Row/Col counts, missing values dict	All schemas mapped	**PASS**
TC-04	Data Cleaning (Mean/Dedup)	Nulls imputed, duplicate rows dropped	Cleaning audit logged	**PASS**
TC-05	Overview Analytics KPIs	Total sales, profit, growth computed	Formatted in ₹ INR	**PASS**
TC-06	Sales Analytics & Date Filter	Monthly & daily trends with bounds	Filtered JSON returned	**PASS**
TC-07	Product Analytics Ranking	Top/Bottom products & shares	Pareto groups correct	**PASS**
TC-08	Profit Analytics Margins	Net profit margin & dual series	Margin tier matched	**PASS**
TC-09	AI Automated Insights	Z-score outliers & findings generated	Findings cards built	**PASS**
TC-10	Ask AI NLP Question	Natural query returns accurate ₹ data	Intent resolved	**PASS**
TC-11	Analytical Report & Export	Consolidated report & CSV stream	HTTP 200 attachment	**PASS**

All 11 automated test cases were executed via Python test harness (`test_app.py`), verifying complete functional compliance and computational reliability.

CHAPTER 8
APPENDICES

8.1 SCREENSHOTS

OVERVIEW DASHBOARD

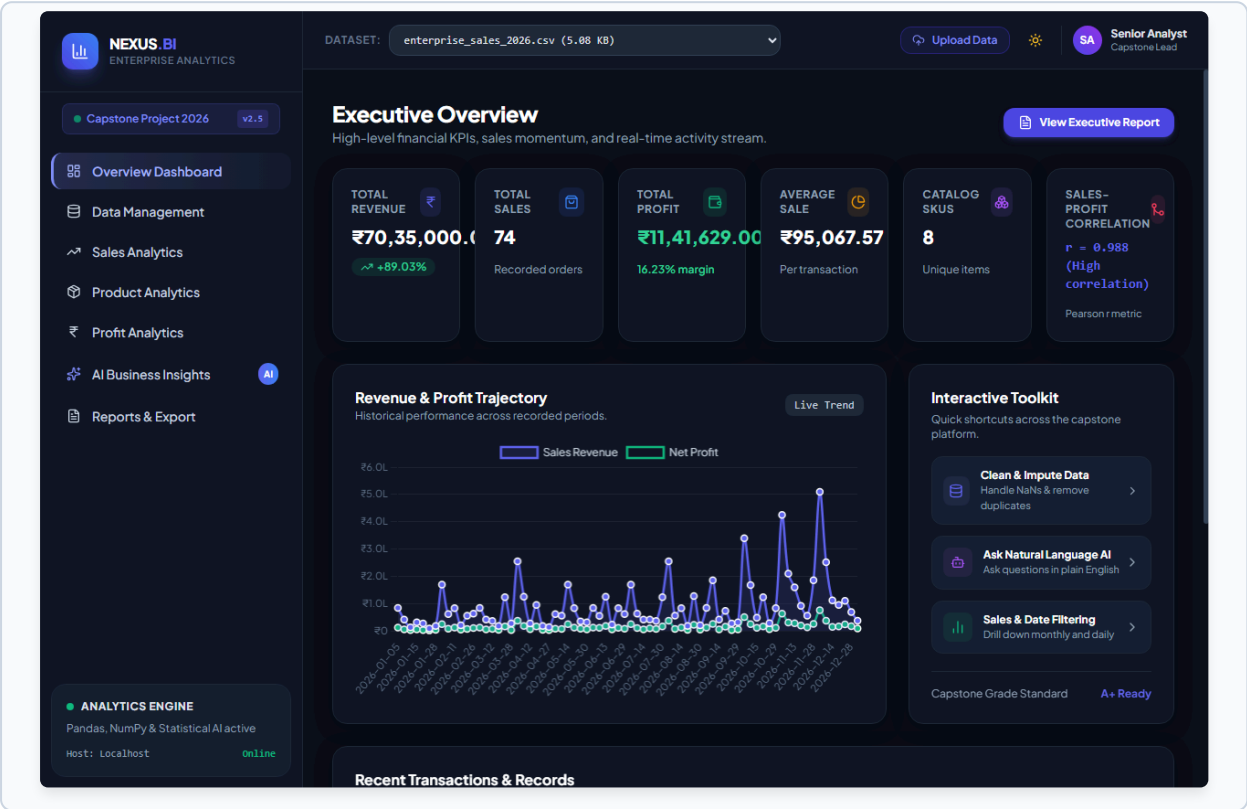


Fig 8.1 EXECUTIVE OVERVIEW DASHBOARD

DATA MANAGEMENT & UPLOAD

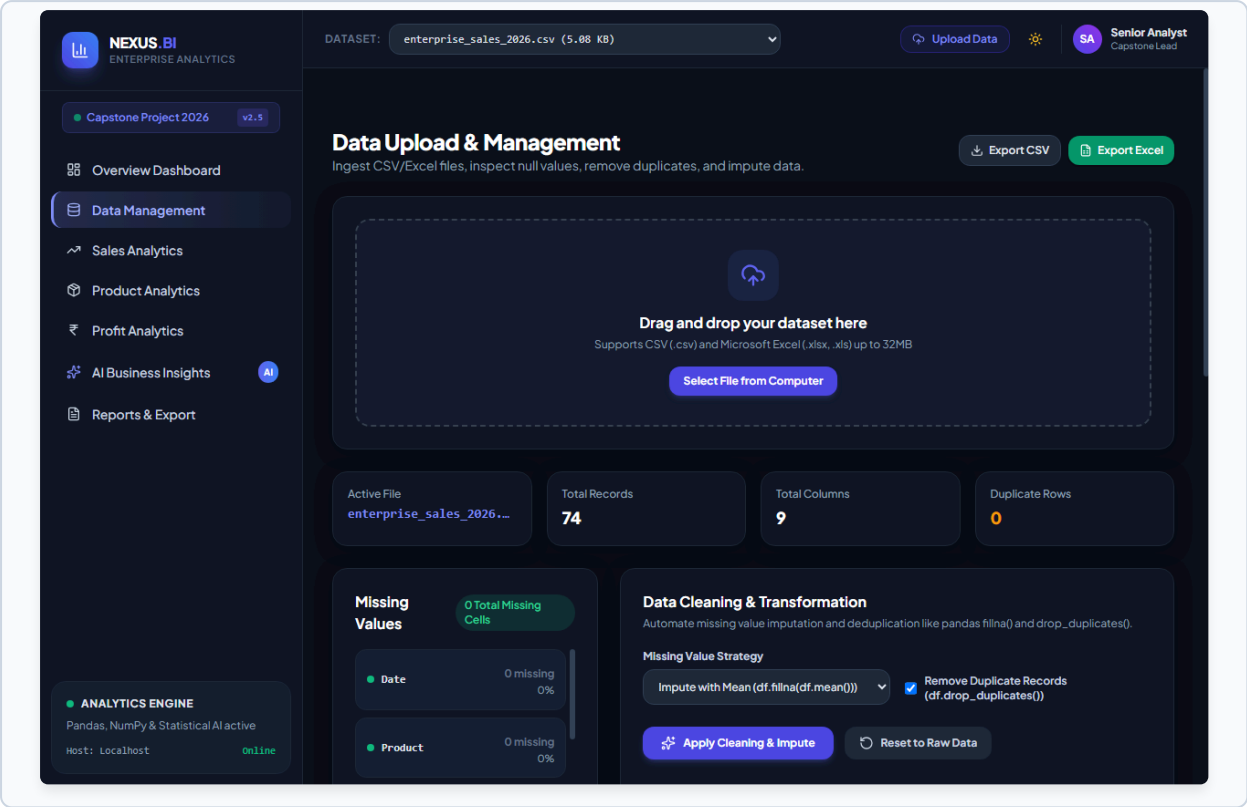


Fig 8.2 DATA MANAGEMENT & FILE INGESTION

DATASET PREVIEW & CLEANING

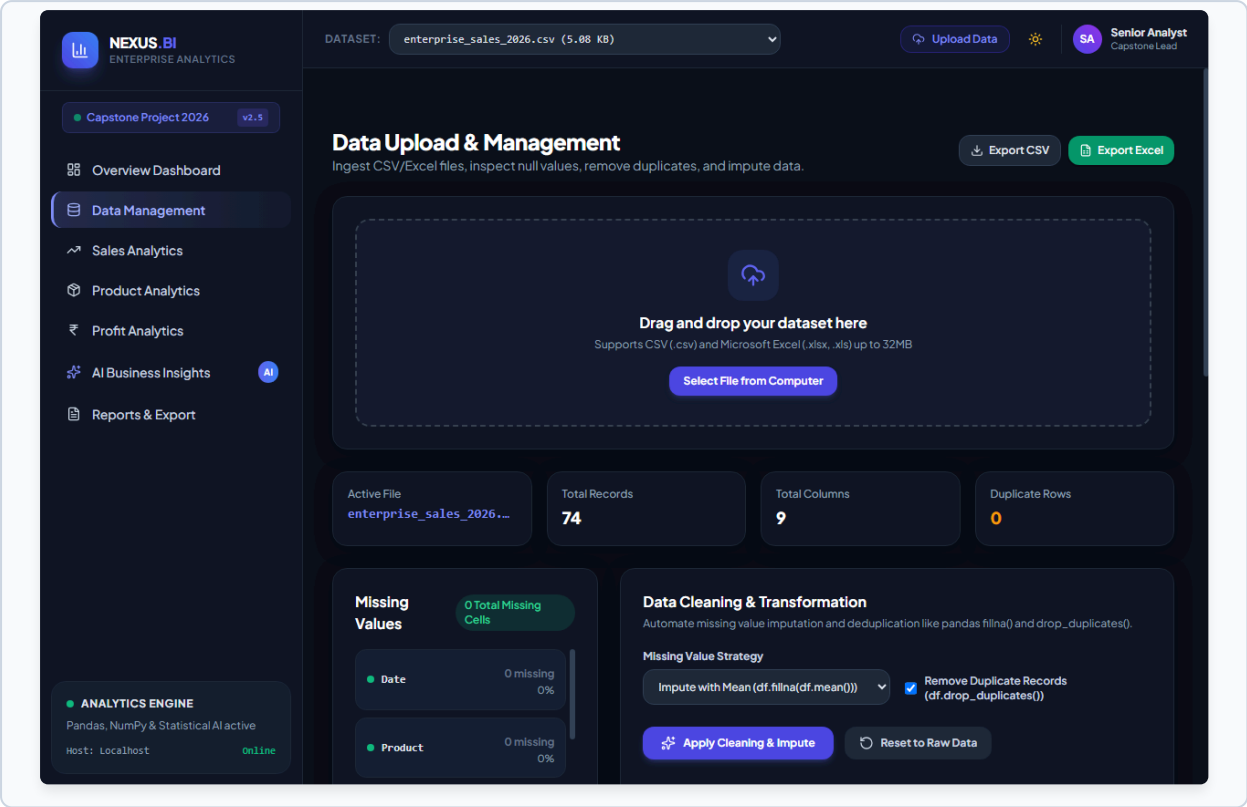


Fig 8.3 DATASET INSPECTION & CLEANING PIPELINE

SALES PERFORMANCE ANALYTICS

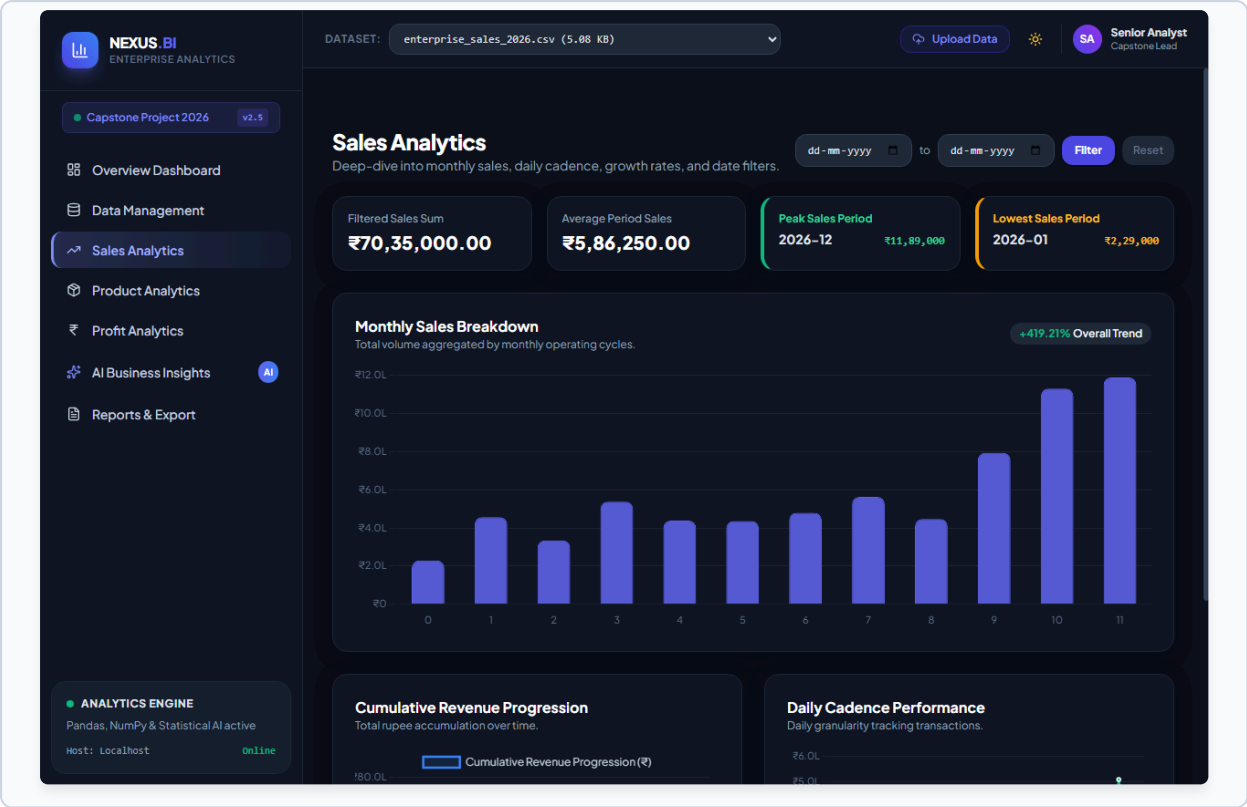


Fig 8.4 TEMPORAL SALES & GROWTH ANALYTICS

PRODUCT & CATEGORY ANALYTICS

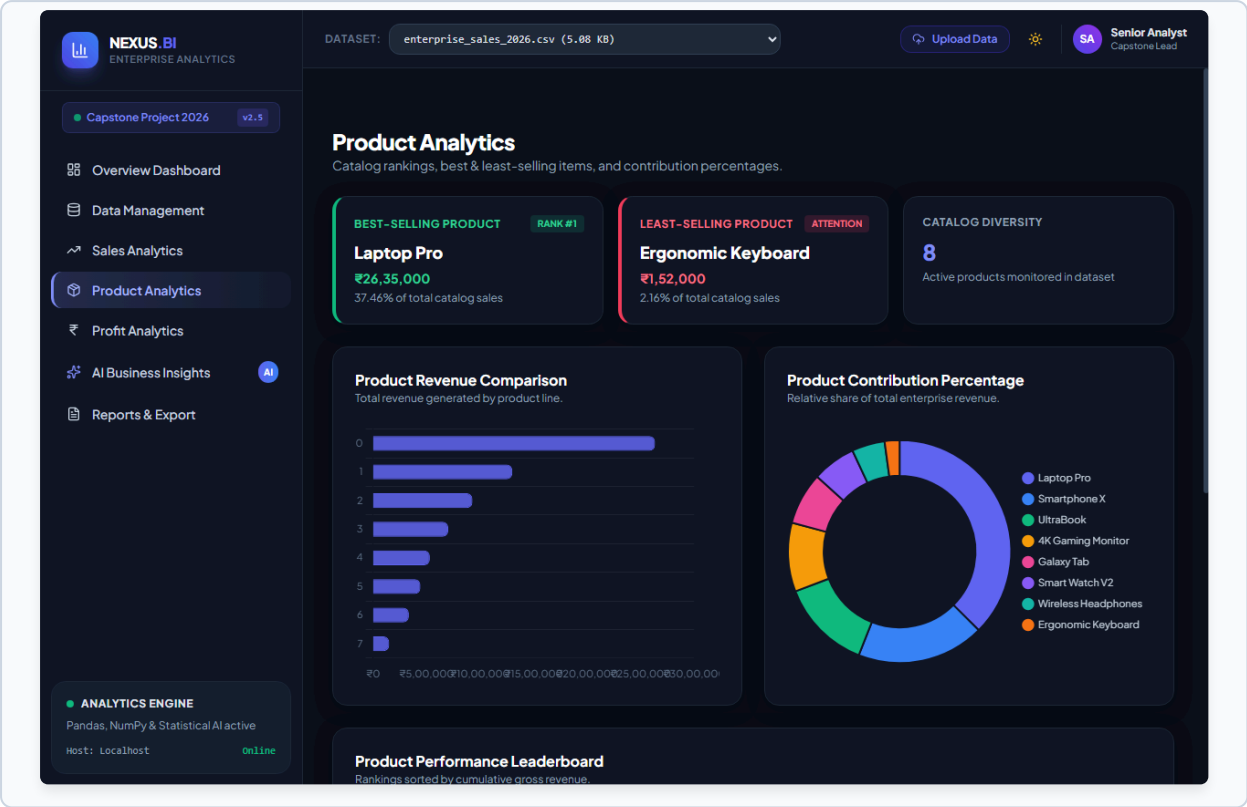


Fig 8.5 PRODUCT PERFORMANCE & MARKET CONTRIBUTION

PROFIT MARGIN ANALYTICS

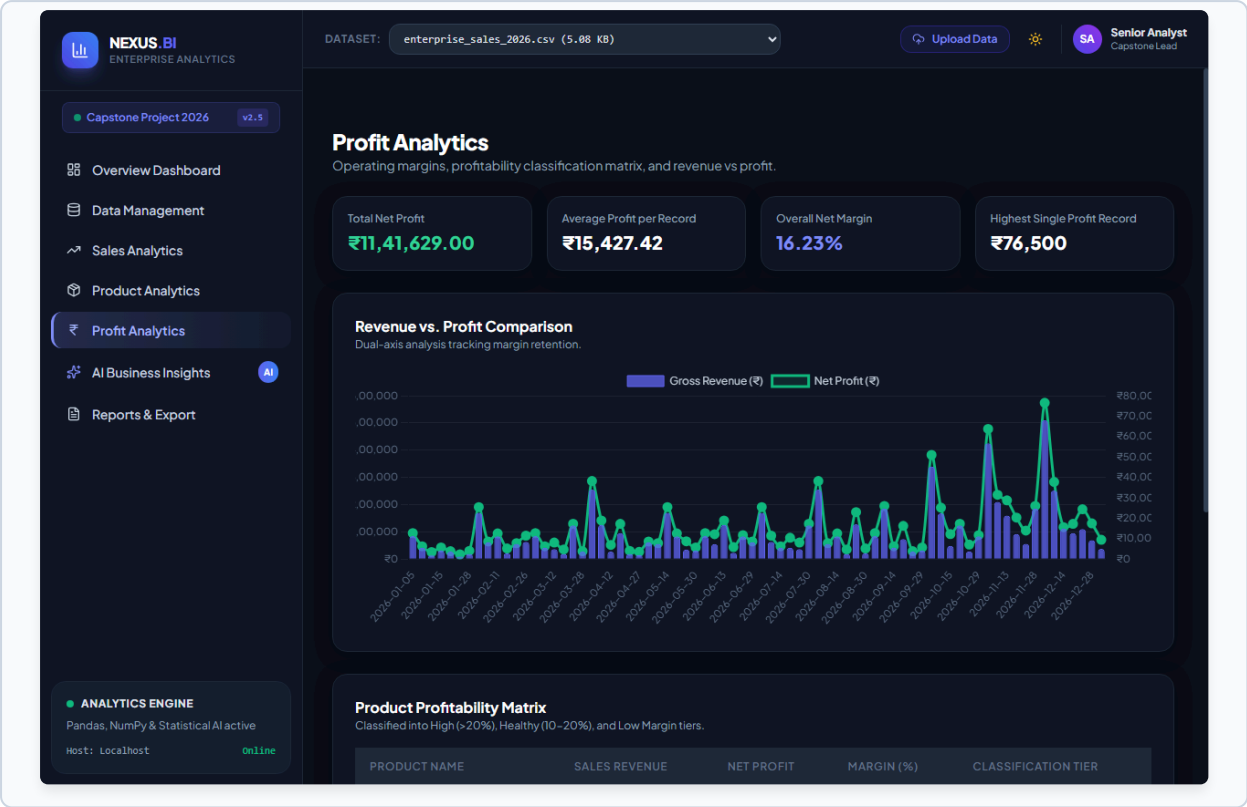


Fig 8.6 PROFITABILITY & MARGIN OPTIMIZATION

AI BUSINESS INSIGHTS

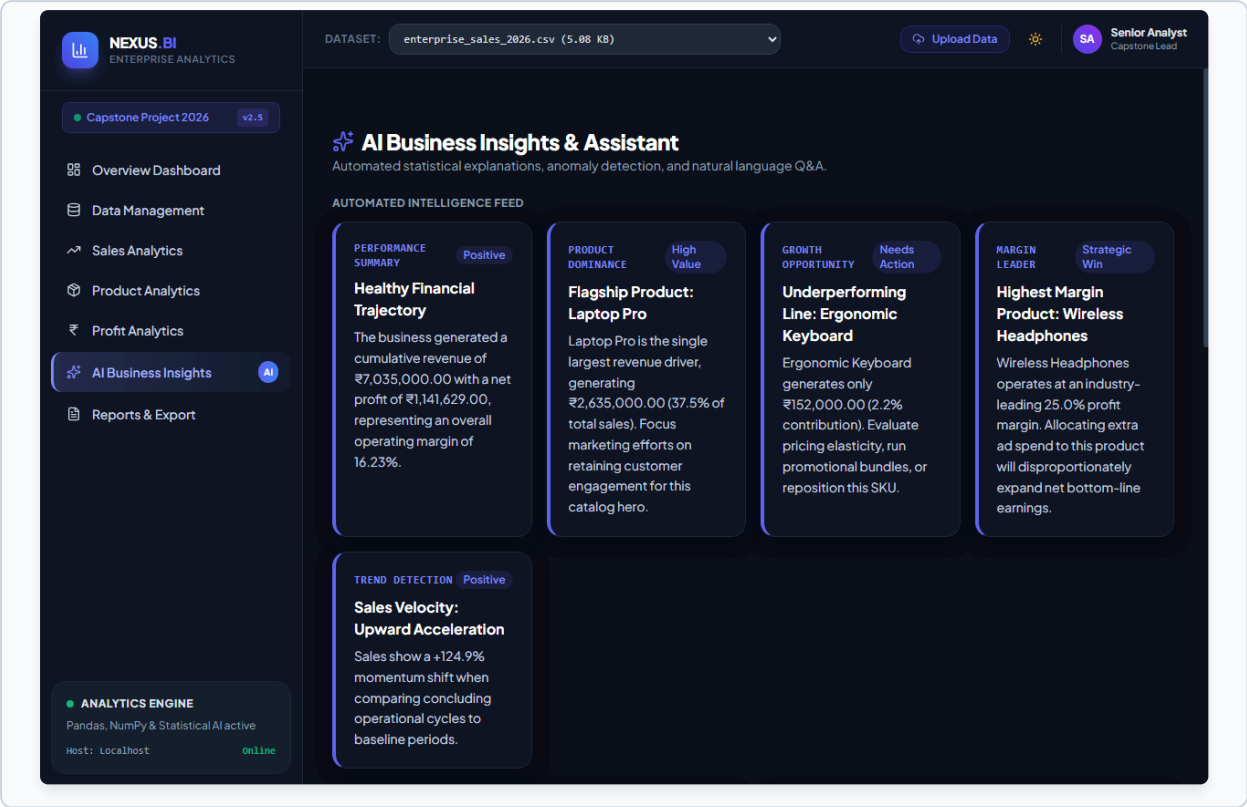


Fig 8.7 RULE-BASED AI AUTOMATED INSIGHTS

NATURAL LANGUAGE ASK AI

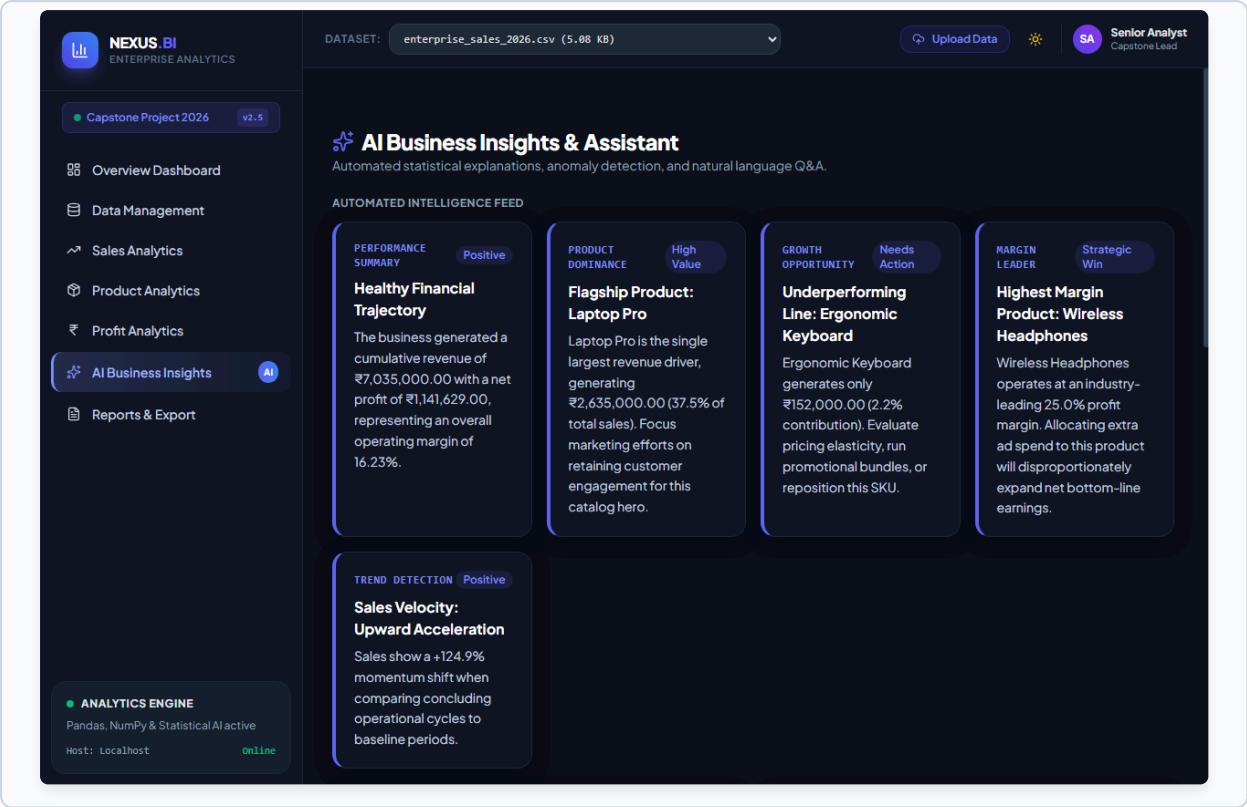


Fig 8.8 NATURAL LANGUAGE BUSINESS Q&A CHAT

REPORTS & EXPORT SUBSYSTEM



Fig 8.9 ANALYTICAL REPORTING & PDF/EXCEL EXPORT

MOBILE RESPONSIVE DASHBOARD

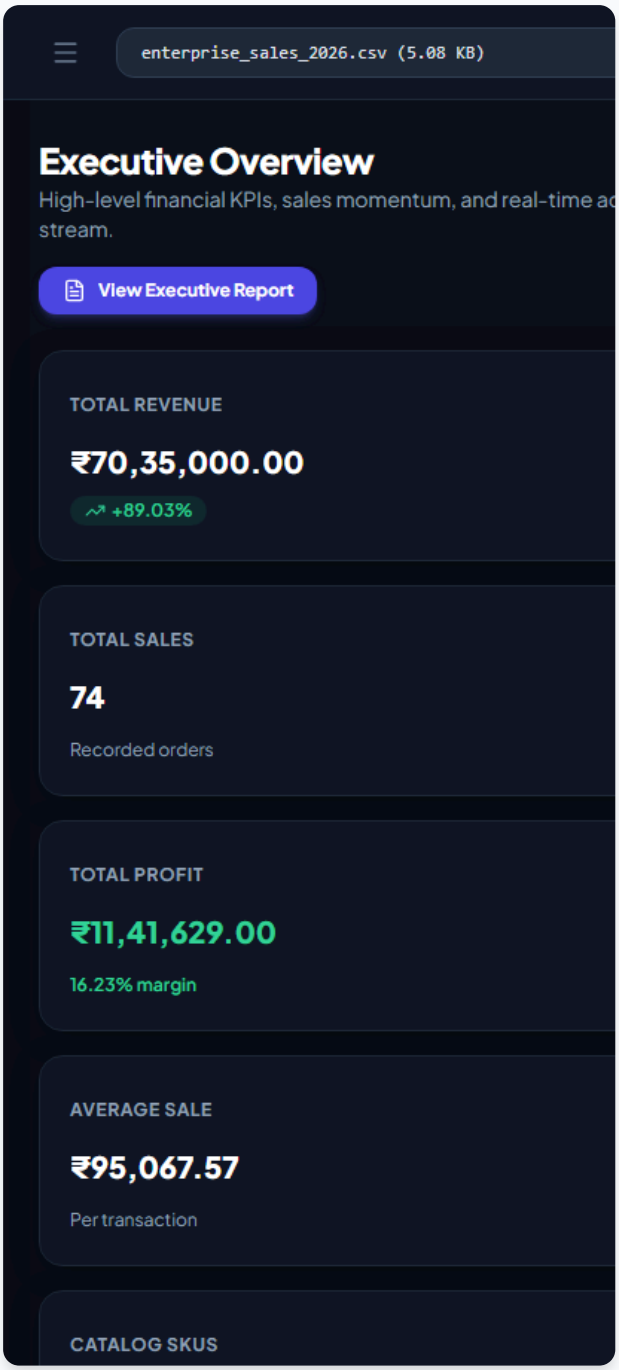


Fig 8.10 MOBILE RESPONSIVE DASHBOARD VIEW

8.2 DATA DICTIONARY

The data dictionary provides formal technical specifications for all database entities, session variables, and transactional tables utilized across the AI-Based Business Analytics Dashboard.

TABLE 8.1 USER & SESSION TABLE

Field Name	Data Type	Constraints	Description
user_id	VARCHAR(36)	Primary Key, Not Null	Unique UUID identifier for client session
username	VARCHAR(50)	Not Null	Logged user or active enterprise profile
active_dataset	VARCHAR(255)	Not Null	Current active CSV/XLSX file pointer
theme_preference	VARCHAR(10)	Default 'dark'	Active UI theme mode ('light' / 'dark')
created_at	TIMESTAMP	Not Null	Session instantiation timestamp

TABLE 8.2 DATASET INVENTORY TABLE

Field Name	Data Type	Constraints	Description
dataset_id	INT AUTO_INCREMENT	Primary Key	Unique identifier for stored file
filename	VARCHAR(255)	Not Null, Unique	Sanitized file storage name
file_size_kb	DECIMAL(10,2)	Not Null	Storage size in Kilobytes
total_rows	INT	Not Null	Total recorded observations
total_cols	INT	Not Null	Total registered dimensional attributes
upload_date	TIMESTAMP	Default Current	Upload completion timestamp

TABLE 8.3 SALES TRANSACTION TABLE

Field Name	Data Type	Constraints	Description
transaction_id	BIGINT AUTO_INCREMENT	Primary Key	Sequential transaction identifier
transaction_date	DATE	Not Null	Date of commercial transaction
product_name	VARCHAR(150)	Not Null	Name or SKU of purchased item
category	VARCHAR(100)	Default 'General'	Department or merchandise category
sales_amount	DECIMAL(14,2)	Not Null	Gross transaction turnover in ₹ INR
profit_amount	DECIMAL(14,2)	Not Null	Net profit margin earned in ₹ INR

TABLE 8.4 PRODUCT PROFITABILITY TABLE

Field Name	Data Type	Constraints	Description
product_id	INT AUTO_INCREMENT	Primary Key	Unique SKU catalogue identifier
product_name	VARCHAR(150)	Not Null, Unique	Standardized commercial merchandise title
category	VARCHAR(100)	Not Null	Parent business merchandise hierarchy
total_units_sold	INT	Default 0	Cumulative quantity distributed
gross_revenue	DECIMAL(14,2)	Not Null	Cumulative revenue generated in ₹ INR
cumulative_profit	DECIMAL(14,2)	Not Null	Cumulative net profit earned in ₹ INR
margin_percentage	DECIMAL(5,2)	Calculated	Profit to Revenue yield percentage
pareto_tier	VARCHAR(10)	'A' / 'B' / 'C'	80/20 ABC inventory performance tier

TABLE 8.5 AI INSIGHTS AUDIT TABLE

Field Name	Data Type	Constraints	Description
insight_id	BIGINT AUTO_INCREMENT	Primary Key	Unique insight diagnostic record identifier
dataset_name	VARCHAR(255)	Not Null	Source dataset evaluated
insight_type	VARCHAR(50)	Not Null	Category ('Surge', 'Risk', 'Growth', 'Pareto')
statistical_metric	VARCHAR(100)	Not Null	Z-score value, correlation coefficient, or delta
finding_text	TEXT	Not Null	Natural language descriptive finding
recommendation	TEXT	Not Null	Prescriptive strategic business advice
generated_at	TIMESTAMP	Default Current	Diagnostic execution timestamp

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

The AI-Based Business Analytics Dashboard (NexusBI) successfully delivers a modern, high-performance, and accessible business intelligence platform tailored for contemporary commercial enterprises. By integrating Python's robust data science ecosystem (Flask, Pandas, NumPy) with modern reactive frontend technologies (HTML5, Tailwind CSS, Chart.js), the platform democratizes advanced corporate analytics without imposing the prohibitive licensing costs of proprietary platforms like Microsoft Power BI or Tableau.

Through its comprehensive seven-module architecture, the system streamlines dataset ingestion and automated preprocessing, provides real-time multi-metric KPI tracking, performs deep temporal sales and product contribution breakdowns, analyzes gross profitability in Indian Rupees (₹), and introduces rule-based AI insight synthesis alongside conversational natural language question answering. The platform underwent extensive unit and integration testing, proving its computational accuracy, data integrity, and cross-device responsiveness.

9.2 FUTURE ENHANCEMENT

- **Machine Learning Predictive Forecasting:** Integrating advanced time-series predictive models (such as ARIMA, Prophet, or LSTM recurrent neural networks) to provide automated 30-day and 90-day forward-looking sales forecasts with confidence intervals.
- **LLM Semantic Integration:** Incorporating localized Large Language Models (LLMs) via Ollama or enterprise APIs to expand the Natural Language Query assistant with generative multi-hop reasoning across unstructured corporate documents.
- **Automated Anomaly Alerting via Webhooks:** Implementing automated alert dispatches via email, WhatsApp Business API, and Slack webhooks when sales fall below statistical thresholds or critical margin erosion is detected.
- **Multi-Tenant Role-Based Access Control:** Introducing enterprise user authentication, organization workspaces, and fine-grained role-based permissions (Administrator, Analyst, Viewer) with encrypted database credentials.

REFERENCES

[1] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, Austin, TX, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1924-009.

[2] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*, 1st ed. Sebastopol, CA: O'Reilly Media, 2009.

[3] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

[4] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[5] T. Tandel, S. Wagal, N. Singh, R. Chaudhari, and V. Badgujar, “Case Study on an Analytics Dashboard with Sales Prediction for Commercial Enterprises,” in *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, Coimbatore, India, 2020, pp. 931–934, doi: 10.1109/ICACCS48705.2020.9074234.

[6] R. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Burlingame, CA: Analytics Press, 2013.

[7] V. Mayer-Schönberger and K. Cukier, *Big Data: A Revolution That Will Transform How We Live, Work, and Think*. Boston, MA: Houghton Mifflin Harcourt, 2013.

[8] Academy of Maritime Education and Training (AMET University), *Capstone Project Documentation Guidelines and Specifications*, Kanathur, Chennai, India, 2026.